

Practical No.1: Write programs to evaluate any expression using Input/Output functions

Practical Significance:

This practical is useful for students to evaluate expressions using different types of operators and Input/output functions. Students will be able to evaluate any type of expressions in the C++ program.

II Industry/Employer Expected Outcome(s)

This practical is expected to develop the following skills as:

1. Develop C++ program using input-output functions.
2. Develop C++ program to evaluate any expressions using various operators.

III Course Level Learning Outcome(s)

Write C++ programs using classes and objects.

IV Laboratory Learning Outcome(s)

Develop program to evaluate expressions using various operators and Input/output functions.

V Relevant Affective Domain related outcome(s)

1. Select proper programming environment in C++.
2. Follow ethical practices.

VI Relevant Theoretical Background

Operators in C++:

An **operator** is a symbol that operates on a value to perform specific mathematical or logical computations. They form the foundation of any programming language. In C++, we have built-in operators to provide the required functionality.

An operator operates the **operands**. For example,

`int c = a + b;`

Here, '+' is the addition operator. 'a' and 'b' are the operands that are being 'added'.

Different types of operators are:

1. Arithmetic Operators
2. Relational Operators
3. Logical Operators
4. Bitwise Operators
5. Assignment Operators
6. Ternary or Conditional Operators

C++ Basic Input/Output:

C++ I/O operation is using the stream concept. Stream is the sequence of bytes or flow of data. It makes the performance fast. If bytes flow from main memory to device like printer, display screen, or a network connection, etc, this is called as **output operation**.

If bytes flow from device like printer, display screen, or a network connection, etc to main memory, this is called as **input operation**.

Header File	Function and Description
<iostream>	It is used to define the cout, cin and cerr objects, which correspond to standard output stream, standard input stream and standard error stream, respectively.
<iomanip>	It is used to declare services useful for performing formatted I/O, such as setprecision and setw.
<fstream>	It is used to declare services for user-controlled file processing.

Standard output stream (cout)

The cout is a predefined object of ostream class.

It is connected with the standard output device, which is usually a display screen. The cout is used in conjunction with stream insertion operator (<<) to display the output on a console.

Standard input stream (cin)

The cin is a predefined object of istream class.

It is connected with the standard input device, which is usually a keyboard.

The cin is used in conjunction with stream extraction operator (>>) to read the input from a console.

Standard end line (endl)

The endl is a predefined object of ostream class.

It is used to insert a new line character and flushes the stream

VII Resources Required

Sr. No	Name Resource	Specification	Quantity	Remarks
1	Computer System	Any desktop or laptop computer with basic configuration	One computer system for each student	
2	Operating system	Windows / LINUX	One for each computer system	
3	Software	Turbo C++ Version 3.0 or any other	One for each computer system	

VIII Precautions to be followed

1. Handle computer system and peripherals with care.
2. Follow safety practices.

IX Exercise:

1. Write a C++ code to evaluate following expression using input output function:
 $y = 5 * x - 5$ where value of x is taken from user. Find the value of y .
2. Write a program to print "hi" msg if entered value is more than 10 otherwise print "bye" msg on output screen. (Use of Relational operator)
3. Write a program to print largest number among two number using Conditional operator.

X C++ code:

Write "C++" Code for above exercise on the blank pages attached at the end of practical.

XI Resources Used

Sr. No.	Name of Resource	Suggested Broad Specification	Quantity
	Turboc ++	3.0	1

XII Result(s)

In this practical we successfully execute program using input and output string.

XIII Conclusion

In this practical we wrote c++ program to get desired output.

XIV Practical Related Questions

Note: Below given are few sample questions for reference. Teacher must design more such questions so as to ensure the achievement of identified CO.

(Note: for all relevant programming exercise use blank pages provided or attach more pages if needed.)

1. Write a C++ program to evaluate the following expressions:
 $X = (-b - (b^2 - 4ac)) / 2a$

2. Complete the following table:

Program Code	Write & justify Output
a) #include<iostream.h> #define PI 3.14159 int main () { float r = 2; float circle; circle= 2 *PI* r; cout << circle; return 0; }	Output :- 12.5664
b) #include <iostream> int main() { int x; x=(3/2) + 2; cout<<"Value of x is : "<<x; return 0; }	output : value of x is : 3
c) #include <iostream> int main() { int x; int y=9; x=y +int(10.0); cout<<"Value of x : "<<x; return 0; }	Output : value of x : 19

(Space for Answers)

Q.1] #include <iostream.h>
 #include <conio.h>
 void main()
 {
 int x,y;
 cout << "Enter value of x:"
 cin >> x;
 y = 5 * x - 5;
 cout << "value of y:" << y << endl;
 getch();
 }

Output 8-

Enter value of x: 2
value of y: 5

Q.2] Ans:-

```
#include <iostream.h>
```

```
#include <conio.h>
```

```
void main()
```

```
{
```

```
int n;
```

```
cout << "Enter a value:";
```

```
cin >> n;
```

```
if (n > 10)
```

```
cout << "Hi" << endl;
```

```
else
```

```
cout << "Bye" << endl;
```

```
getch();
```

```
}
```

Output 8-

Enter a value : 1

Bye

Enter a value : 12

Hi

Q.3] Ans:-

```
#include <iostream.h>
```

```
#include <conio.h>
```

```
void main () {
```

```
int a, b;
```

```
cout << "Enter 2 numbers:";
```

```

cin >> a >> b;
(a > b) ? cout << "Greater number is: " << a :
cout << "Smallest "
number is: " << b << endl;
getch();
?

```

Output :-

```

Enter 2 numbers : 34 45
Smallest number is : 34
Greater number is : 45

```

• Practical Related Question

Q.1] Ans :-

```

#include <iostream.h>
#include <conio.h>
void main()
{
    int a, b, c, x;
    cout << "Enter 3 numbers: ";
    cin >> a >> b >> c;
    x = (c - b - c * b * b - 4 * a * c) / 2 * a;
    cout << "\n x: " << x;
    getch();
}

```

Output :-

```

Enter 3 numbers: 1 2 4
x: 5

```


XV References/Suggestions for further reading

1. <https://www.geeksforgeeks.org/program-evaluate-simple-expressions>
2. <https://www.javatpoint.com/cpp-expression>

XVI Assessment Scheme

Performance Indicators		Weightage
Process Related: 30 Marks		60 %
1	Logic formation	10%
2	Appropriate use of arithmetic expressions, operators and basic Input/output function.	20%
3	Debugging ability	20%
4	Follow ethical practices.	10%
Product Related: 20 Marks		40%
5	Expected Output	20%
6	Submitting the Manual in time	10%
7	Answer to sample questions	10%
Total (50 Marks)		100 %

Marks Obtained			Dated signature of Teacher
Process Related (30)	Product Related (20)	Total (50)	

Practical No.2: Write programs using-

- Scope resolution operator
- Memory management operator
- Manipulators

I Practical Significance:

This practical is useful for students to access the global variable, creation of dynamic memory allocation and displaying output in proper format. Students will be able to use different C++ operators in programming.

II Industry/Employer Expected Outcome(s)

This practical is expected to develop the following skills as:

1. Develop C++ program using Scope resolution operator.
2. Develop C++ program to using memory management operators.
3. Develop C++ program to display the output in proper format using different manipulators.

III Course Level Learning Outcome(s)

Write C++ programs using classes and objects.

IV Laboratory Learning Outcome(s)

Develop C++ program using special type of operators.

V Relevant Affective Domain related outcome(s)

1. Select proper programming environment in C++.
2. Follow ethical practices.

VI Relevant Theoretical Background

1. Scope resolution Operator:

The scope resolution operator is used to reference the global variable or member function that is out of scope. Therefore, we use the scope resolution operator to access the hidden variable or function of a program.

The operator is represented as the double colon :: symbol.

For example, when the **global and local variable or function** has the same name in a program, and when we call the variable, by default it only accesses the **inner or local variable** without calling the global variable. In this way, it hides the global variable or function. To overcome this situation, we use the scope resolution operator to fetch a

Uses of the scope resolution Operator

- It is used to access the hidden variables or member functions of a program.
- It defines the member function outside of the class using the scope resolution.
- It is used to access the static variable and static function of a class.
- The scope resolution operator is used to override function in the Inheritance.

2. Memory Management Operator:

Memory management is a process of managing computer memory, assigning the memory space to the programs to improve the overall system performance.

C++ defines unary operators such as new and delete to perform the tasks, i.e., allocating and freeing the memory.

• New operator

A new operator is used to create the object while a delete operator is used to delete the object. When the object is created by using the new operator, then the object will exist until we explicitly use the delete operator to delete the object. Therefore, we can say that the lifetime of the object is not related to the block structure of the program.

`pointer_variable = new data-type`

The above syntax is used to create the object using the new operator.

In the above syntax, 'pointer_variable' is the name of the pointer variable, 'new' is the operator, and 'data-type' defines the type of the data.

Example 1:

```
int *p;  
p = new int;
```

In the above example, 'p' is a pointer of type int.

• Delete operator

When memory is no longer required, then it needs to be deallocated so that the memory can be used for another purpose. This can be achieved by using the delete operator, as shown below:

`delete pointer_variable;`

In the above statement, 'delete' is the operator used to delete the existing object, and 'pointer_variable' is the name of the pointer variable.

In the previous case, we have created pointer 'p' by using the new operator, and can be deleted by using the following statements:

`delete p;`

3. Manipulators:

Manipulators are helping functions that can modify the input/output stream. It does not mean that we change the value of a variable, it only modifies the I/O stream using insertion (<<) and extraction (>>) operators.

Manipulators are special functions that can be included in the I/O statement to alter the format parameters of a stream. Manipulators are operators that are used to format the data display. To access manipulators, the file **iomanip.h** should be included in the program.

- **endl C++ Manipulator:**

The word 'endl' in C++, a programming language, stands for end of line. Furthermore, the use of the endl C++ manipulators takes place to move the cursor to the beginning of the next line. Moreover, its working is similar to the '\n' escape sequence.

```
cout<<"object"<<endl<<"Oriented"<<endl<<"Programming";
```

- **setw Manipulator:**

The word 'setw' in C++ stands for set width. Furthermore, the use of the setw C++ manipulator takes place to set the output's field width on the output device. Moreover, by default, the displaying or printing of the output takes place right-justified within the specified field of setw C++ manipulators.

The use of the setw manipulator takes place to set the width of the output in a program. Furthermore, it takes up an argument 'n', the width of the field in which the displaying of the output is to take place. Moreover, the output in the field, by default, is right-aligned.

The general syntax of setw manipulator is as follows:

```
setw(n)
```

The 'n' indicates the field width that is the number of columns and it happens to be an integer value. Also, if the specified field width is less in comparison to the width of the output data's width that is to be displayed, then the effect of setw manipulator will not take place. Moreover, there will be a normal displaying or printing of the output.

The setw C++ manipulator is also a component of "**iomanip.h**" header file. Furthermore, the inclusion of this header file must take place in the program to use setw manipulator.

Some important manipulators in <iomanip> are:

- **setw (val):** It is used to set the field width in output operations.
- **setfill (c):** It is used to fill the character 'c' on output stream.
- **setprecision (val):** It sets val as the new value for the precision of floating-point values.
- **setbase(val):** It is used to set the numeric base value for numeric values.
- **setiosflags(flag):** It is used to set the format flags specified by parameter mask.
- **resetiosflags(m):** It is used to reset the format flags specified by parameter mask.

Object Oriented Programming Using C++ (313304)

X Resources Required

Sr. No	Name Resource	Specification	Quantity	Remarks
1	Computer System	Any desktop or laptop computer with basic configuration	One computer system for each student	
2	Operating system	Windows / LINUX	One for each computer system	
3	Software	Turbo C++ Version 3.0 or any other	One for each computer system	

XI Precautions to be followed

1. Handle computer system and peripherals with care.
2. Follow safety practices.

XII Exercise:

1. Write a C++ Program which show the use of function outside the class using scope resolution operator.
2. Write a program to display the message "Welcome to the world of C++" using manipulators.
3. Write a program to create the memory using new operator and free the created memory using delete operator.

X C++ code:

Write "C++" Code for above exercise on the blank pages attached at the end of practical.

XI Resources Used

Sr. No.	Name of Resource	Suggested Broad Specification	Quantity
	Turbo c++	3.0	1

XII Result(s)

In this practical we use scope resolution operator, memory management operator and manipulator and get desired output.

XIII Conclusion

In this practical we write various C++ codes to get desired output.

Object Oriented Programming Using C++ (313304)

XIV Practical Related Questions

Note: Below given are few sample questions for reference. Teacher must design more such questions so as to ensure the achievement of identified CO.

(Note: for all relevant programming exercise use blank pages provided or attach more pages if needed.)

1. Write a C++ program to access the global variable using scope resolution operator.
2. Format the following statement using manipulators.

(Rate=412345 period=35 year=2024)

3. Complete the following table:

Program Code	Write & Justify Output
<pre>a) #include <iostream> int x; // Global x int main() { int x = 10; // Local x cout << "Value of global x is" << x; cout << "\nValue of local x is" << x; return 0; }</pre>	value of global x is 0 value of local x is 10
<pre>b. #include <iostream> class A { public: // Only declaration void fun(); }; // Definition outside class using :: void A::fun() { cout << "fun() called"; } int main() { A a; a.fun(); return 0; }</pre>	Fun called.
<pre>c. #include <iostream> #include <iomanip> int main (void) { int a,b; a = 200; b = 300;</pre>	

Object Oriented Programming Using C++ (313304)

```
cout << setw (5) << a << setw (5) << b << endl;
cout << setw (6) << a << setw (6) << b << endl;
cout << setw (7) << a << setw (7) << b << endl;
cout << setw (8) << a << setw (8) << b << endl;
return 0;
|
```

```
200 300
200 300
200 300
200 300
```

(Space for Answers)

Q.1] →

```
#include <iostream.h>
#include <conio.h>
int x = 40;
int main ()
{
    int x = 10;
    cout << "Global : " << x << endl;
    cout << "Local : " << x << endl;
    getch();
}
```

output :- global : 40
Local : 10

Q.2] →

```
#include <iostream.h>
#include <conio.h>
int main () {
    int rate = 412345;
    int period = 95;
    int year = 2024;
    cout << "Rate : " << setw (10) << rate << endl;
    cout << "Period : " << setw (10) << period << endl;
    cout << "Year : " << setw (10) << year << endl;
    getch();
}
```


Exercise

Q.17 Ans:-

```
#include <iostream.h>
```

```
#include <conio.h>
```

```
class Rectangle
```

```
{
```

```
public:
```

```
int height, base;
```

```
void area (int h, int b);
```

```
};
```

```
void Rectangle::area (int h, int b)
```

```
{
```

```
height = h;
```

```
base = b;
```

```
cout << "Area is = " << height * base;
```

```
}
```

```
void main ()
```

```
{
```

```
Rectangle r;
```

```
r.area (5, 3);
```

```
getch ();
```

```
}
```


Q.2] Ans:-

```
#include <iostream.h>
```

```
#include <conio.h>
```

```
#include <iomanip.h>
```

```
void main ()
```

```
{ clrscr();
```

```
cout << "setw (50) << "welcome to the world  
of c++ " << endl;
```

```
getch();
```

```
}
```

Q.3] Ans:-

```
#include <iostream.h>
```

```
#include <conio.h>
```

```
void main ()
```

```
{
```

```
int * point - int = new int [45];
```

```
float * point - float = new float [45.45];
```

```
clrscr();
```

```
cout << "Point - int";
```

```
cout << "Point - float";
```

```
delete point - int;
```

```
delete point - float;
```

```
getch(); }
```

Practical No.4: Write programs to show use of classes and objects to define the function inside the class

I Practical Significance:

The classes and objects help to represent real life entity with different attributes and related Member functions.

II Industry/Employer Expected Outcome(s)

This practical is expected to develop the following skills as:

- i. Define and use classes and objects.
- ii. Define the member function inside the class.

III Course Level Learning Outcome(s)

Write C++ programs using classes and objects.

IV Laboratory Learning Outcome(s)

Implement classes and objects to define the function inside class.

V Relevant Affective Domain related outcome(s)

1. Select proper programming environment in C++.
2. Follow ethical practices.

VI Relevant Theoretical Background

• **Class:** Class is a user defined data type, which holds its own data members and associated member functions, which can be accessed and used by creating an instance of that class.

• **Object:** An Object is an instance of a Class. When a class is defined, no memory is allocated but when it is instantiated (i.e. an object is created)

class ClassName

{
Access specifier: Data members;
Member functions()

//can be private, public or protected
// Variables to be used
//Methods to access data members

{

 //body of the function

}

}; //Class ends with semicolon

Declaring Objects: When a class is defined, only the specification for the object is defined; no memory or storage is allocated. To use the data and access functions defined in the class, you need to create objects.

Syntax:

className **ObjectName;**

eg.

Student std;

Accessing data members and member functions: The data members and member functions of class can be accessed using the dot('.') operator with the object.

For example if the name of object is std and you want to access the member function with the name getData() then you will have to write std.getData() .

Member Functions in Classes: There are two ways to define functions that belongs to a class:

- Inside class definition
- Outside class definition

Define Member Function Inside the class:

Syntax: class
 className

```
{  
    //other members declaration  
    return type function_Name(list of parameters)  
    {  
        // body of function  
    }  
    //other members declaration  
};
```

Class Methods: A member function of a class can also be defined inside the class. However, when a member function is defined inside the class, the class name and the scope resolution operator are not specified in the function header. Moreover, the member functions defined inside a class definition are by default inline functions called.

In the following example, we define a function inside the class, and we name it "myMethod".

Note: You access methods just like you access attributes; by creating an object of the class and using the dot syntax (.):

Inside Example

```
class MyClass  
{  
    // The class  
    public:                      // Access specifier  
    void myMethod()  
    {  
        // Method/function defined inside the class  
        cout << "Hello World!";  
    }  
};
```



```

int main()
{
    MyClass myObj; // Create an object of MyClass
    myObj.myMethod(); // Call the method
    return 0;
}

```

VII Resources Required

Sr. No	Name Resource	Specification	Quantity	Remarks
1	Computer System	Any desktop or laptop computer with basic configuration	One computer system for each student	
2	Operating system	Windows / LINUX	One for each computer system	
3	Software	Turbo C++ Version 3.0 or any other	One for each computer system	

VIII Precautions to be followed

1. Handle computer system and peripherals with care.
2. Follow safety practices.

IX Exercise:

1. Write syntax for define member function inside the class.
2. Write a program to define a class student having data members name and roll no. Accept and display data for one object. Define the member function inside the class.
3. Write a program to calculate square of number in which define class and define member function inside the class as per requirement.
4. Write a program to define a class student having data members name and roll no. Accept and display data for one object. Define the member function inside the class.
5. What is the difference between struct and class in C++?

X C++ code:

Write "C++" Code for above exercise on the blank pages attached at the end of practical.

XI Resources Used

Sr. No.	Name of Resource	Suggested Broad Specification	Quantity
1.	Computer system	laptop or desktop	1
2.	Operating system	Windows / LINUX	1
3.	Software	Turbo C++	1

XII Result(s)

In this practical we learn how to create class and that object. How to define function inside the class to get desired output.

XIII Conclusion

In this program we use different class & define function inside class.

XIV Practical Related Questions

Note: Below given are few sample questions for reference. Teacher must design more such questions so as to ensure the achievement of identified CO.

(Note: for all relevant programming exercise use blank pages provided or attach more pages if needed.)

1. Define a class Room with data members length, breadth and height. Member function calculate_area () and calculate_volume(). Calculate the area and volume of room. Define the member function inside the class.

2. Define a class mean in which assign two numbers in assign member function (i.e. assign(4,8)) passed value from main function and define assign member function inside the class and display the mean of two number on output screen.

3. Complete the following table:

Program Code	Write & justify Output
<pre> a) #include <iostream> class Student { public: int id; //data member (also instance variable) string name; //data member (also instance variable) }; int main() { Student s1; //creating an object of Student s1.id = 201; s1.name = "Sonoo Jaiswal"; cout << s1.id << endl; cout << s1.name << endl; return 0; </pre>	201 Sonoo Jaiswal

Object Oriented Programming Using C++ (313304)

```
b. #include <iostream>
class Student {
public:
    int id; //data member (also instance variable)
    string name; //data member (also instance variable)
    void insert(int i, string n)
    {
        id = i;
        name = n;
    }
    void display()
    {
        cout << id << " " << name << endl;
    }
};

int main(void) {
    Student s1; //creating an object of Student
    Student s2; //creating an object of Student
    s1.insert(201, "Sonoo");
    s2.insert(202, "Nakul");
    s1.display();
    s2.display();
    return 0;
}
```

201 Sonoo
202 Nakul

(Space for Answers)

Exercise:

Q.1] Ans:-

```
class class_name
```

```
{
```

```
    Public:
```

```
    // other members declaration
```

```
    return Type function name (parameter list)
```

```
{
```

```
    // function body
```

```
}
```

```
    // other members declaration
```

```
};
```

Q.2] Ans:- #include <iostream.h>

```
class Student
```

```
    string name;
```

```
    int roll no;
```

```
    Public:
```

```
    void accept ()
```



```
{
```

```
        cout << "Enter name:" << endl;
```

```
        cin >> name;
```

```
        cout << "Enter Rollno:" << endl;
```

```
        cin >> rollno;
```

```
}
```

```
    void display ()
```

```
{
```

```
        cout << "Name: " << name << endl;
```

```
        cout << "Rollno: " << rollno << endl;
```

```
}
```

```

};
int main ()
{
    student s;
    s.accept();
    s.display();
    return 0;
}

```

Q.3] Ans:-

```

#include <iostream.h>
#include <conio.h>
class square
{
    public:
        int number;
        void number ()
        {
            cin >> number;
        }
        void cal-sq()
        {
            int sq = number * number;
            cout << sq;
        }
}

```



```

{
};
void main ()
{
    student s;
    s.acceptData();
    s.displayData();
    getch();
}

```

Q. 5] Ans:-
Class

structure

- | | |
|---|---|
| 1] Members of a class are private by default. | 1] Members of a structure are public by default. |
| 2] It is declared using the class keyword. | 2] It is declared using the struct keyword. |
| 3] It is normally used for data abstraction. | 3] It is normally used for the grouping of different datatypes. |
| 4] Syntax:-
class class_name { data-member; member-function; } | 4] Syntax:-
struct structure_name { structure 1; ... 2; 3; } |

```

};
int main ()
{
    Square s;
    s.number ();
    s.cal-sq ();
    return 0;
}

```

Q.4] Ans 8-

```

#include <iostream.h>
class student
{
    public:
        string name;
        int rollno;
        void accept data ()
        {
            cout << "Enter name:";
            cin >> name;
            cout << "Enter rollno:";
            cin >> rollno;
        }
        void display data ()
        {
            cout << "Name:" << name << endl;
            cout << "Roll no:" << rollno;
        }
    };

```


Example:

```
1.#include<iostream.h>
class Class_name{

    public:

    return_type Method_name(); // method outside class definition
};
```

// Outside the Class using scope resolution operator

```
return_type Class_name :: Method_name() {

    // body of member function

}
```

2. #include <iostream>

```
class Sq {
public:
    int a;
    int square(); // Declaring function square with no argument and having
                  // return type 'int'.
};

int Sq::square()
{
    return a * a;
}
```

VII Resources Required

Sr. No	Name Resource	Specification	Quantity	Remarks
1	Computer System	Any desktop or laptop computer with basic configuration	One computer system for each student	
2	Operating system	Windows / LINUX	One for each computer system	
3	Software	Turbo C++ Version 3.0 or any other	One for each computer system	

I Practical Significance:

The classes and objects help to represent real life entity with different attributes and related Member functions.

II Industry/Employer Expected Outcome(s)

This practical is expected to develop the following skills as:

- Define and use classes and objects.
- Define the member function Outside the class.
- Use of scope resolution operator.

III Course Level Learning Outcome(s)

Write C++ programs using classes and objects.

IV Laboratory Learning Outcome(s)

Implement classes and objects to define the function outside class..

V Relevant Affective Domain related outcome(s)

- Select proper programming environment in C++.
- Follow ethical practices.

VI Relevant Theoretical Background

Member Functions in Classes: There are two ways to define functions that belongs to a class:

- Inside class definition
- Outside class definition

Define Member Function Outside class definition:

Syntax: return type className :: functionName(list of parameters)

```
{
    //body of function
}

MyClass myObj; // Create an object of MyClass
myObj.myMethod(); // Call the method
return 0;
}
```


VIII Precautions to be followed

1. Handle computer system and peripherals with care.
2. Follow safety practices.

IX Exercise:

1. Write syntax for define member function Outside the class.
2. Write a C++ program to declare a class "staff" having data members name, basic salary, DA, HRA and calculate gross salary. Accept and display data for one staff.

Where DA=74.5% of basic

- i. HRA= 30% of basic.
- ii. Gross_salary=basic+HRA+DA

X C++ code:

Write "C++" Code for above exercise on the blank pages attached at the end of practical.

XI Resources Used

Sr. No.	Name of Resource	Suggested Broad Specification	Quantity
1.	Turbo c++	version 3.0	1

XII Result(s)

In this practical I can learn how to define function outside the class with the scope resolution operator.

XIII Conclusion

We written c++ codes to define the function outside class and execution them.

XIV Practical Related Questions

Note: Below given are few sample questions for reference. Teacher must design more such questions so as to ensure the achievement of identified CO.

(Note: for all relevant programming exercise use blank pages provided or attach more pages if needed.)

1. Write a program to find area of circle such that the class circle must have three functions namely:
 - a) read() to accept the radius from user.

b)compute() for calculating the area

c)display() for displaying the result.(Use Scope resolution operator)

2. Define a class complex with data members real and imaginary, member function read() and write(). Write a program to perform the addition of two complex number and display the result.

3. Complete the following table:

Program Code	Write & justify Output
<pre> a) #include <iostream> struct X { int a, b; // member function declaration int add(); }; // global variable int a = 10; // define member function outside class int X::add() { return a + b; } int main() { int answer; X xobject; xobject.a = 1; xobject.b = 2; answer = xobject.add(); cout << xobject.a << " + " << xobject.b << " = " << answer << endl; } </pre>	output - 1 + 2 = 12
<pre> b. #include <iostream> // Declaration of the class class MyClass { public: void myFunction(); // Function prototype declaration // inside the class }; // Definition of the function outside the class void MyClass::myFunction() { std::cout << "This is myFunction() defined outside the class." << std::endl; } int main() { MyClass obj; obj.myFunction(); // Call the function return 0; } </pre>	output:- This is my-function() defined outside the class;


```

}
cout << "The area of the circle is: " << area;
}

int main ()
{
    Circle c;
    c.read ();
    c.compute ();
    c.display ();
    return 0;
}

```

Output :-

Q.2] Ans :-

```

#include <iostream.h>
#include <conio.h>
class complex
{
    float real, imaginary;
public:
    void read ()
    {
        cout << "Enter real part : ";
        cin >> real;
        cout << "Enter imaginary part : ";
        cin >> imaginary;
    }
    void write ()
    {
        cout << real;
        if (imaginary >= 0)

```

Output :-

Enter the name of the staff : John

Enter the basic salary : 50000

Staff Name : John

Basic salary : 50000

Dearness Allowance (DA) : 37250

House Rent Allowance (HRA) : 15000

Gross salary : 102250

XIV. Practical Related Questions.

Q.1. #include <iostream.h>

class circle {

private:

double radius;

double area;

public:

void read();

void compute();

void display();

};

void circle::read()

{

cout << "Enter the radius of the circle:";

cin >> radius;

}

void circle::compute()

{

area = 3.14159 * radius * radius;

}

void circle::display()


```
cin >> basic_salary;
```

```
?
```

```
void staff::calculate()
```

```
{
```

```
DA = 0.745 * basic_salary;
```

```
HRA = 0.30 * basic_salary;
```

```
gross_salary = basic_salary + DA + HRA;
```

```
}
```

```
void staff::display()
```

```
{
```

```
cout << "Staff Name:" << name << endl;
```

```
cout << "Dearness Allowance (DA):"
```

```
<< DA << endl;
```

```
cout << "House Rent Allowance (HRA):"
```

```
<< HRA << endl;
```

```
cout << "Gross Salary:" << gross_salary << endl;
```

```
}
```

```
int main()
```

```
{ clrscr();
```

```
staff s;
```

```
s.read();
```

```
s.calculate();
```

```
s.display();
```

```
getch();
```

```
return 0;
```

```
}
```

Exercise:-

Q.1] Ans:-

return-type

class-name:: function_name (parameters - IPst)

{

// function body

}

Q.2] Ans:-

#include <iostream.h>

#include <conio.h>

class staff

{

private:

char name[50];

float basic_salary;

float DA;

float HRA;

float gross_salary;

public:

void read();

void calculate();

void display();

};

void staff::read()

{

cout << "Enter the name of the staff:";

cin >> name;

cout << "Enter the basic salary:";


```
cout << "+" << imaginary << "i" << endl;
else
cout << "-" << -imaginary << "i" << endl;
}
Complex add (Complex c)
{
    Complex temp;
    temp.real = real + c.real;
    temp.imaginary = imaginary + c.imaginary;
    return temp;
}

int main()
{
    Complex c1, c2, c3;
    cout << "Enter 1st Complex number:";
    c1.read();
    cout << "Enter 2nd Complex number:";
    c2.read();
    c3 = c1.add(c2);
    cout << "Sum of 2 complex numbers: \n";
    c3.write();
    getch();
}
```

Practical No.6: Write programs to implement Inline functions.

Practical Significance:

The classes and objects help to represent real life entity with different attributes and related member functions. The use of inline functions facilitates faster execution of the program

Industry/Employer Expected Outcome(s)

An inline function that reduces the execution time of a program

Course Level Learning Outcome(s)

Develop C++ programs using classes and objects.

Laboratory Learning Outcome(s)

Implement programs using Inline functions

Relevant Affective Domain related outcome(s)

1. Select proper programming environment in C++.
2. Follow ethical practices.

Relevant Theoretical Background

Inline Function: C++ inline function is concept that is commonly used with classes. If a function is inline, the compiler places a copy of the code of that function at each point where the function is called at compile time.

A function definition in a class definition is an inline function definition, even without the use of the **inline** keyword.

To inline a function, place the keyword **inline** before the function name and define the function before any calls are made to the function. The compiler can ignore the inline qualifier in case defined function is more than a line.

Declaring inline function:

Syntax:

```
class className
{
    //other members declaration public:
    inline return type FunctionName(list of parameters)
    {
        //body of inline function
    }
    //other members declaration
```


VII Resources Required

Sr. No	Name Resource	Specification	Quantity	Remarks
1	Computer System	Any desktop or laptop computer with basic configuration	One computer system for each student	
2	Operating system	Windows / LINUX	One for each computer system	
3	Software	Turbo C++ Version 3.0 or any other	One for each computer system	

VIII Precautions to be followed

1. Handle computer system and peripherals with care.
2. Follow safety practices.

IX Exercise:

1. Write a C++ program to create a class "Number" having data members n1 and n2 and perform mathematical operations like addition, subtraction, multiplication and division on two numbers using inline functions.

X C++ code:

1. Write "C++" Code for above exercise on the blank pages attached at the end of practical.

XI Resources Used

Sr. No.	Name of Resource	Suggested Broad Specification	Quantity
1-	Turbo c++	Version 3.0	1

XII Result(s)

In this practical I have learned about implementation of inline function.

XIII Conclusion

In this practical we successfully executed the program implement inline function.

XIV Practical Related Questions

Note: Below given are few sample questions for reference. Teacher must design more such questions so as to ensure the achievement of identified CO.

(Note: for all relevant programming exercise use blank pages provided or attach more pages if needed.)

- Write a C++ program to calculate area of Rectangle using Inline function.
- Complete the following table:

Program Code	Write & justify Output
<pre>a) #include <iostream> using namespace std; inline int exp(int x, int y, int z) { return (x + y) * z; } int main() { cout << exp(4,5,7) << endl; cout << exp(4,5,6) << endl; cout << exp(4,7,5) << endl; cout << exp(7,4,6) << endl; return 0; }</pre>	Output :- 63 64 65 66
<pre>b) #include <iostream> using namespace std; class Sample { public: inline void myfun(); }; void inline Sample::myfun() { cout << "Hello from inline function"; } int main() { Sample S; S.myfun(); return 0; }</pre>	Output :- Hello from Inline function

(Space for Answers)

Exercise

Q.17 Ans:-

```
#include <iostream.h>
#include <conio.h>
class number
{
private:
double n1;
double n2;
public:
void read();
inline double add();
inline double sub();
inline double multi();
inline double div();
};

void number::read()
{
cout << "Enter the first number:";
cin >> n1;
cout << "Enter the second number:";
cin >> n2;
}

inline double Number::add()
{
return n1+n2;
}

inline double Number::sub()
{
return n1-n2;
}

inline double Number::multi()
{
return n1*n2;
}
```

```

}
inline double divi()
{
    if (n2 != 0)
        return n1/n2;
    else {
        cout << "Error : Division by 0";
    }
}
void main()
{
    clrscr();
    Number num;
    num.read();
    cout << "Addition : " << num.add() << endl;
    cout << "Subtraction : " << num.sub() << endl;
    cout << "Multiplication : " << num.multi << endl;
    cout << "Division : " << num.divi << endl;
    getch();
}

```

Practical Related Question

Q.17 Ans:-

```

#include <iostream.h>
#include <conio.h>
class Rectangle {
    double length, width;
public:
    void read() {
        cout << "Enter length and width:";
        cin >> length >> width;
    }
    void display() {

```



```
Box(int cap){
    capacity = cap;
}

friend void show();
};

void show()
{
    Box b(10);
    cout<<"Value of capacity
    is:"<<b.capacity<<endl;
}

int main(int argc, char const *argv[])
{
    show();
    return 0;
}
```

Q.11 Ans8-

(Space for Answers)

```
#include <iostream.h>
#include <conio.h>
class calculation {
    float a, b;
public:
    void setvalues (float x, float y) {
        a=x;
        b=y;
    }
    friend void displayResults (calculation);
};

void displayResults(calculation calc){
    cout << "Addition : "<< calc.a + calc.b << endl;
    cout << "Subtraction : "<< calc.a - calc.b << endl;
    cout << "Multiplication : "<< calc.a * calc.b << endl;
    cout << "Division : "<< calc.a / calc.b << endl;
}

void main {
    calculation calc;
    float x=12, y=4;
    calc.setvalues (x, y);
    displayResults (calc);
}
```


Practical No.7: Write program to Implement Friend Function using

- Two different classes
- External Function

I Practical Significance:

The classes and objects help to represent real life entity with different attributes and related member functions.
The use of friend function allows non-member functions to access the private and protected data of the class.

II Industry/Employer Expected Outcome(s)

Friend function is widely used in cases when two or more classes contain the interrelated members relative to other parts of the program.

III Course Level Learning Outcome(s)

Develop C++ programs using constructors.

IV Laboratory Learning Outcome(s)

Implement programs using Friend functions

V Relevant Affective Domain related outcome(s)

1. Select proper programming environment in C++.
2. Follow ethical practices.

VI Relevant Theoretical Background

- **Friend Function:** A friend function of a class is defined outside that class' scope but it has the right to access all private and protected members of the class. Even though the prototypes for friend functions appear in the class definition, friends are not member functions. **friend** keyword is used to declare the function as friend of class.

- Declaring friend

function

Syntax:

```
class className
```

```
{
```

```
    //other members declaration
```



```
cout << "Area:" << area() << endl;
```

```
}
```

```
};
```

```
void main ()
```

```
{
```

```
clrscr();
```

```
Rectangle r;
```

```
r.read();
```

```
r.display();
```

```
getch();
```

```
}
```



XII Result(s)

In this practical, I have learned about to access friend function using two different classes and external function.

XIII Conclusion

We wrote various C++ code for implement friend function successfully.

XIV Practical Related Questions

Note: Below given are few sample questions for reference. Teacher must design more such questions so as to ensure the achievement of identified CO.

(Note: for all relevant programming exercise use blank pages provided or attach more pages if needed.)

1. WAP to declare a class calculation. Display addition, subtraction, multiplication, division of two numbers. Use friend function.
2. Complete the following table:

Program Code	Write & justify Output
a) How many member functions are there in this C++ class excluding constructors and destructors? <pre> class Box { int capacity; public: void print(); friend void show(); bool compare(); friend bool lost(); }; </pre>	There are 4 member function 1) void print() 2) friend void show() 3) friend bool lost() 4) bool compare();
b) What will be the output of the following C++ code? <pre> #include <iostream> #include <string> using namespace std; class Box { int capacity; public: </pre>	output:- value of capacity is: 10

Q.1] Ans:

```
#include <iostream.h>
```

```
#include <conio.h>
```

```
class Swap {
```

```
    int a, b;
```

```
public:
```

```
    void setvalues (int x, int y) {
```

```
        a = x;
```

```
        b = y;
```

```
    }
```

```
    void displayvalues () {
```

```
        cout << "a:" << a << "b:" << b << endl;
```

```
    }
```

```
    friend void exchange (Swap &);
```

```
};
```

```
void exchange (Swap &s) {
```

```
    int temp = s.a;
```

```
    s.a = s.b;
```

```
    s.b = temp;
```

```
}
```

```
int main () {
```

```
    Swap s;
```

```
    s.setvalues (10, 20);
```

```
    cout << "Before swapping:";
```

```
    s.displayvalues ();
```

```
    exchange (s);
```

```
    cout << "After swapping:";
```

```
    s.displayvalues ();
```

```
    getch ();
```

```
}
```

Q.2] Ans:-

```
#include <iostream.h>
#include <conio.h>
class Test2;
class Test1
{
    int marks1;
    public:
    void setMarks1(int m) { marks1 = m; }
    friend float calculateAverage(Test1, Test2);
};
float calculateAverage(Test1 t1, Test2 t2)
{
    return (t1.marks1 + t2.marks2) / 2.0;
}
void main()
{
    clrscr();
    Test1 t1;
    Test2 t2;
    t1.setMarks1(85);
    t2.setMarks(90);
    cout << "Average : " << calculateAverage
    (t1, t2);
    getch();
}
```


Practical No.8: Write program to Implement-

- Static Data Member
- Static Member Function

I Practical Significance:

A typical use of static members is for recording data common to all objects of a class. Static data member as a counter to store the number of objects of a particular class type that are created.

II Industry/Employer Expected Outcome(s)

Static data member in C++ can be accessed anywhere in the program after the declaration of class either using the class instance or using scope resolution, class name, and variable name

III Course Level Learning Outcome(s)

Develop C++ programs using constructors.

IV Laboratory Learning Outcome(s)

Implement programs using Static Members and Static Member Functions

V Relevant Affective Domain related outcome(s)

1. Select proper programming environment in C++.
2. Follow ethical practices.

VI Relevant Theoretical Background

- Static data members are class members that are declared using static keywords. A static member has certain special characteristics which are as follows:
Only one copy of that member is created for the entire class and is shared by all the objects of that class, no matter how many objects are created.
It is initialized before any object of this class is created, even before the main starts.
It is visible only within the class, but its lifetime is the entire program.

- Declaring Static Data Member-

Syntax:

static data_type data_member_name;

- The static member functions are special functions used to access the static data members or other static member functions. A member function is defined using the static keyword. A static

Practical No.8: Write program to Implement-

- Static Data Member
- Static Member Function

I Practical Significance:

A typical use of static members is for recording data common to all objects of a class. Static data member as a counter to store the number of objects of a particular class type that are created.

II Industry/Employer Expected Outcome(s)

Static data member in C++ can be accessed anywhere in the program after the declaration of class either using the class instance or using scope resolution, class name, and variable name

III Course Level Learning Outcome(s)

Develop C++ programs using constructors.

IV Laboratory Learning Outcome(s)

Implement programs using Static Members and Static Member Functions

V Relevant Affective Domain related outcome(s)

1. Select proper programming environment in C++.
2. Follow ethical practices.

VI Relevant Theoretical Background

- Static data members are class members that are declared using static keywords. A static member has certain special characteristics which are as follows:
Only one copy of that member is created for the entire class and is shared by all the objects of that class, no matter how many objects are created.
It is initialized before any object of this class is created, even before the main starts.
It is visible only within the class, but its lifetime is the entire program.
- Declaring Static Data Member-
Syntax:
`static data_type data_member_name;`
- The static member functions are special functions used to access the static data members or other static member functions. A member function is defined using the static keyword. A static

XIV Practical Related Questions

Note: Below given are few sample questions for reference. Teacher must design more such questions so as to ensure the achievement of identified CO.

(Note: for all relevant programming exercise use blank pages provided or attach more pages if needed.)

1. Write a Program to calculate weight of object at different planets using formula $\text{weight} = m \times G$. Where m =mass of object G =gravitational force. Declare g as static member variable.
2. Complete the following table:

Program Code	Write & justify Output
a) Which is correct syntax to access the static member functions with class name?	Output :- class-name::function name. (parameter)
b) What will be the output of the following C++ code? <pre> class Test { private: static int x; public: static void fun() { cout <<< ++x <<< " "; } }; int Test::x = 20; void main() { Test x; x.fun(); x.fun(); }</pre>	Output :- 21 22
c) What will be the output of the following C++ code? <pre> class Test { public: Test() { cout <<< "Test's Constructor is Called " <<< endl; } }</pre>	Output :- test constructor is called Result constructor is called.


```

};

class Result
{
    static Test a;
public:
    Result()
    {
        cout << "Result's Constructor is Called " << endl;
    }
};

void main()
{
    Result b;
}

```

Exercise

(Space for Answers)

```

Q.1] #include <iostream.h>
#include <conio.h>
class SimpleInterest {
float principal, duration;
static float rate_of_interest;
public:
void setvalues (float p, float d) {
    principal = p;
    duration = d;
}
void calculate And display () {
    float interest = (principal * duration * rate_of_interest) / 100;
    cout << "Simple Interest : " << interest << endl;
}
};
float SimpleInterest :: rate_of_interest = 5.0;
void main () {
    SimpleInterest si;
    float p, d;
    cout << "Enter Principal amount: ";
    cin >> p;
    cout << "Enter Duration (years) : ";
    cin >> d;
    si.setvalues (p, d);
    si.calculate And display ();
    getch();
}

```


Practical

Q.17 Ans:-

```
#include <iostream.h>
```

```
#include <conio.h>
```

```
class weightcalculator {
```

```
float mass;
```

```
static float g;
```

```
public:
```

```
void setmass (float m) { mass=m; }
```

```
static void set Gravity (float gravity) { g=gravity; }
```

```
void display weight () { cout << "weight : " <<  
mass * g << " N" << endl; }
```

```
};
```

```
float weightcalculator :: g=9.8;
```

```
void main () {
```

```
weightcalculator wc;
```

```
float m;
```

```
cout << "Enter mass of object (Kg) : ";
```

```
cin >> m;
```

```
wc.setmass(m);
```

```
cout << "select planet (1 for Earth, 2 for  
Mars, 3 for Jupiter) : ";
```

```
void choice;
```

```
cin >> choice;
```

```
switch (choice) {
```

```
case 1: weight calculator :: set Gravity (9.8);  
break;
```

```
case 2: weight calculator :: set Gravity (3.7);
```

```
break ;  
Case 3 : weight calculator :: Set Gravity (24.9);  
break ;  
default : cout << "Invalid choice!" << endl;  
return 1;  
}  
wc.displayweight();  
getch();  
}
```



Practical No.9: Write programs to create Array of Objects.

I Practical Significance:

The classes and objects help to represent real life entity with different attributes and related member functions.

Array of objects are used to represent the data of similar type.

II Industry/Employer Expected Outcome(s)

The array of objects represent storing multiple objects in a single name. Reduce the time and memory by storing the data in a single variable.

III Course Level Learning Outcome(s)

Develop C++ programs using constructors.

IV Laboratory Learning Outcome(s)

Write/ Compile/ debug/ Execute simple C++ program using classes and array of objects.

V Relevant Affective Domain related outcome(s)

1. Select proper programming environment in C++.
2. Follow ethical practices.

VI Relevant Theoretical Background

Array of objects: Arrays of variables of type "class" is known as "Array of objects".

Declaring Array of

Objects: Syntax:

className ObjectArrayName[size];

eg.

Student std[5];

Accessing data members and member functions: The data members and member functions of class can be accessed using the dot('.') operator with the object.

For example if the name of object is *std[2]* and you want to access the member function with the name *getData0* then you will have to write *std[2].getData0*.

VII Resources Required

Sr. No	Name Resource	Specification	Quantity	Remarks
1	Computer System	Any desktop or laptop computer with basic configuration	One computer system for each student	
2	Operating system	Windows / LINUX	One for each computer system	
3	Software	Turbo C++ Version 3.0 or any other	One for each computer system	

VIII Precautions to be followed

1. Handle computer system and peripherals with care.
2. Follow safety practices.

IX Exercise:

Write a C++ program to declare class 'Account' having data members as Account_No and Balance. Accept this data for 10 accounts and display data of Accounts having balance greater than 10000.

X C++ code:

1. Write "C++" Code for above exercise on the blank pages attached at the end of practical.

XI Resources Used

Sr. No.	Name of Resource	Suggested Broad Specification	Quantity
1.	Operating System	Windows	1

XII Result(s)

Hence we learn about write a program to create array of object.

XIII Conclusion

Hence we observe about write a program to create array of object.

XIV Practical Related Questions

Note: Below given are few sample questions for reference. Teacher must

Maharashtra State Board of Technology (K Scheme)

Mahara

design more such questions so as to ensure the achievement of identified CO.

(Note: for all relevant programming exercise use blank pages provided or attach more pages if needed.)

1. Write a Program to declare a class birthday having data member day, month and year. Accept this info for object using pointer to array of object and display it.
2. Complete the following table:

Program Code	Write & justify Output
1) If array of objects is declared as given below, which is the limitation on objects? Class_name arrayName[size];	class.name arrayname [size];
2) What will be the output of the following C++ code? <pre>#include<iostream> using namespace std; class Employee { int id; char name[30]; public: void getdata(); void putdata(); }; void Employee::getdata() { cout << "Enter Id : "; cin >> id; cout << "Enter Name : "; cin >> name; } void Employee::putdata() { cout << id << " "; cout << name << " "; cout << endl; } int main() {</pre>	<id> <name>


```

Employee emp[30];
int n, i;
cout << "Enter Number of Employees - ";
cin >> n;
for(i = 0; i < n; i++)
    emp[i].getdata();

cout << "Employee Data - " << endl;

for(i = 0; i < n; i++)
    emp[i].putdata();
}

```

Exercise

(Space for Answers)

```

Q.1] #include <iostream.h>
#include <conio.h>
class Account {
    int Account-No;
    Float Balance;
    Public:
    void getdata() {
        cin >> Account-No >> Balance;
    }
    void display-Data() {
        if (Balance > 0000) {
            cout << "Account-Number:" << Account-No << "Balance:"
            << Balance << endl;
        }
    }
};

int main() {
    Account accounts[10];
    for (int i=0; i<10; i++) {
        accounts[i].getdata();
    }
    for (int i=0; i<10; i++) {
        accounts[i].get-displaydata();
    }
    getch();
}

```


Practical 8-

Q Ans 8-

```
#include <iostream.h>
```

```
class Birthday {
```

```
int day, month, year;
```

```
public:
```

```
void getdata () {
```

```
cin >> day >> month >> year;
```

```
}
```

```
void displaydata () const {
```

```
cout << day << "/" << month << "/" << year << endl;
```

```
}
```

```
};
```

```
int main () {
```

```
int n;
```

```
cout << "Enter number of birthdays: ";
```

```
cin >> n;
```

```
Birthday * birthday = new Birthday [n];
```

```
for (int i=0; i<n; i++) {
```

```
cout << "Enter details for birthday " << i+1 << " " << endl;
```

```
birthday [i].getdata ();
```

```
}
```

```
cout << "\n Birthday information: " << endl;
```

```
for (int i=0; i<n; i++)
```

```
{
```

```
birthday [i].displaydata ();
```

```
}
```

```
delete () birthday;
```

```
getch();
```

```
}
```


Practical No.10: Write programs for –

- Default Constructor
- Parameterized Constructor
- Copy Constructor
- Multiple Constructor in one class

I Practical Significance:

The classes and objects help to represent real-life entity with different attributes and related member functions.
The use of constructor functions facilitates initialization of instance variables.

II Industry/Employer Expected Outcome(s)

Constructor is to set the initial values of the object's attributes or perform any action that needs to be done for each object.

III Course Level Learning Outcome(s)

Develop C++ programs using constructors.

IV Laboratory Learning Outcome(s)

Write/ Compile/ debug / Execute simple C++ program using constructors and destructors.

V Relevant Affective Domain related outcome(s)

- a. Select proper programming environment in C++.
- b. Follow ethical practices.

VI Relevant Theoretical Background

- **Constructor:** Constructors are special member functions which perform initialization of every object. The Compiler calls the Constructor whenever an object is created. Constructors initialize values to instance variables after storage is allocated to the object.

Constructors are of three types:

1. Default Constructor
2. Parameterized Constructor
3. Copy Constructor

- **Declaring constructor function: Syntax:**
class className

```
{  
    //other members
```

Sr. No.
1
2
3


```

declaration public:
    className(list of parameters)
    {
        //body of constructor function
    }
    //other members declaration
};
    
```

• **Default Constructor In C++**

Syntax of Default Constructor-

```

class ClassName {
public:
    ClassName(); // Default constructor declaration
};
    
```

• **Parameterized Constructor In C++**

Syntax of Parameterized Constructor

```

class ClassName {
public:
    ClassName(Type1 parameter1, Type2 parameter2, ...); // Parameterized
    //constructor declaration
};
    
```

• **Copy Constructor In C++**

Syntax of Copy Constructor

```

class ClassName {
public:
    ClassName(const ClassName& obj); // Copy constructor declaration
};
    
```

VII Resources Required

Sr. No	Name Resource	Specification	Quantity	Remarks
1	Computer System	Any desktop or laptop computer with basic configuration	One computer system for each student	
2	Operating system	Windows /LINUX	One for each computer system	
3	Software	Turbo C++ Version 3.0 or any other	One for each computer system	

VIII Precautions to be followed

1. Handle computer system and peripherals with care.
2. Follow safety practices.

IX Exercise:

1. WAP to implement default constructor that initializes num1 and num2 as 10 and 20 and prints the values of num1 and num2.

Object Oriented Programming Using C++(313304)

- Define a class student which contain member variables as rollno ,name and course. Write a program using constructor as "Computer Engineering" for course . Accept this data for objects of class and display the data.

X C++ code:

- Write "C++" Code for above exercise on the blank pages attached at the end of practical.

XI Resources Used

Sr. No.	Name of Resource	Suggested Broad Specification	Quantity
1.	Software	Turbo C++	1

XII Result(s)

Hence we learn about write program for default constructor, parameterized, copy, multiple constructor in one class.

XIII Conclusion

Hence we observe about write program for default constructor, parameterized, copy, multiple constructor in one class.

XIV Practical Related Questions

Note: Below given are few sample questions for reference. Teacher must design more such questions so as to ensure the achievement of identified CO.

(Note: for all relevant programming exercise use blank pages provided or attach more pages if needed.)

- WAP to declare a class student having datamembers as name and percentage. Write constructor to initialise these data members. Accept and display data for one object.
- WAP to declare class time having data members as hrs, min, sec. Write a constructor to accept data and display for two objects.
- Complete the following table:

Program Code	Write & justify Output
1) What will be the output of the following C++ code? <pre> #include <iostream> #include <string> using namespace std; class A{ mutable int a; public: A(){ cout<<"Default constructor called\n"; } A(const A& a){ cout<<"Copy Constructor called\n"; } }; int main(int argc, char const *argv[]) { A obj; A a1 = obj; A a2(obj); } </pre>	Default constructor called Copy constructor called Copy constructor called.
2) What will be the output of the following C++ code? <pre> #include <iostream> #include <string> using namespace std; class A{ int a; public: A(int i){ a = i; } void assign(int i){ a = i; } int return_value(){ return a; } } </pre>	5

```
};
int main(int argc, char const
*argv[])
{
A obj;
obj.assign(5);
cout<<obj.return_value();
}
```

(Space for Answers)

Exercise

Q.1]

```
#include <iostream.h>
#include <conio.h>
class Example {
int num1;
int num2;
public:
Example () {num1(10), num2(20);}
void print values () const {
cout << "num1 " << endl;
cout << "num2 " << endl;
}
int main() {
Example obj;
obj.print values ();
getch();
}
```


Q.2] Ans:-

```
#include <iostream.h>
#include <conio.h>
#include <string.h>
class student {
    int rollno;
    char name[50];
    string Course;
public:
    Student () { Course ("E-electronic Engineering"); }
    void getdata () {
        cout << "Enter Roll - numbers:";
        cin >> rollno;
        cin.ignore();
        cout << "Enter name:";
        cin.getline (name, 50);
    }
    void display data () const {
        cout << "Roll Number : " << rollno << endl;
        cout << "Name : " << name << endl;
        cout << "Course : " << Course << endl;
    }
};

void main () {
    Student s;
    s.getdata ();
    s.displaydata ();
    getch ();
}
```

Practical :-

```
1] #include <iostream.h>
#include <string.h>
class student {
    char name [50];
    float percentage;
public:
    student (const char* n, float p) {
        strcpy (name, n, size of (name)-1);
        name [size of (name)-1] = '\0';
        percentage = p;
    }
    void displaydata () const {
        cout << "Name : " << name << endl;
        cout << "percentage : " << percentage << "%";
    }
};
void main ()
{
    char student name [50];
    float student percentage;
    cout << "Enter Name:";
    cin >> student name;
    cout << "Enter percentage :";
    cin >> student percentage;
    student student1 (student name, student percentage);
    student1 . displaydata ();
    getch ();
}
```


Q.2] Ans:-

```
#include <iostream.h>
#include <conio.h>
class Time {
    int hrs, min, sec;
public:
    Time (int h, int m, int s) : hrs(h), min(m),
    sec(s) {}
    void display Time () const {
        cout << hrs << " " << min << " " << sec <<
        endl;
    }
};

void main () {
    Time time1 (10, 30, 45);
    Time time2 (7, 15, 30);
    time1.display Time ();
    time2.display Time ();
    getch ();
}
```

Practical No.11: Write programs for –

- Single level Inheritance
- Multilevel Inheritance

I Practical Significance:

The use of inheritance shows the reusability of the existing class properties and deriving a new class with additional properties. Multiple Inheritance is a feature of C++ where a class can inherit from more than one classes.

II Industry/Employer Expected Outcome(s)

Constructor is to set the initial values of the object's attributes or perform any action that needs to be done for each object.

III Course Level Learning Outcome(s)

Implement Inheritance in C++.

IV Laboratory Learning Outcome(s)

Write/ Compile/ debug / Execute simple C++ program using multilevel inheritance.

V Relevant Affective Domain related outcome(s)

3. Select proper programming environment in C++.
4. Follow ethical practices.

VI Relevant Theoretical Background

Inheritance: It is the process of inheriting properties of objects of one class by objects of another class. The class which inherits the properties of another class is called Derived or Child or Sub class and the class whose properties are inherited is called Base or Parent or Super class. When a single class is derived from a single parent class, it is called Single inheritance. It is the simplest of all inheritance

Types of Inheritance:

1. Single Inheritance
2. Multiple Inheritance
3. Multilevel Inheritance
4. Hierarchical Inheritance
5. Hybrid Inheritance

Syntax of Single Inheritance:

```
class base_classname
{
    properties;
    member
    functions;
};
class derived_classname : visibility_mode base_classname
{
    properties;
    member
    functions;
};
```

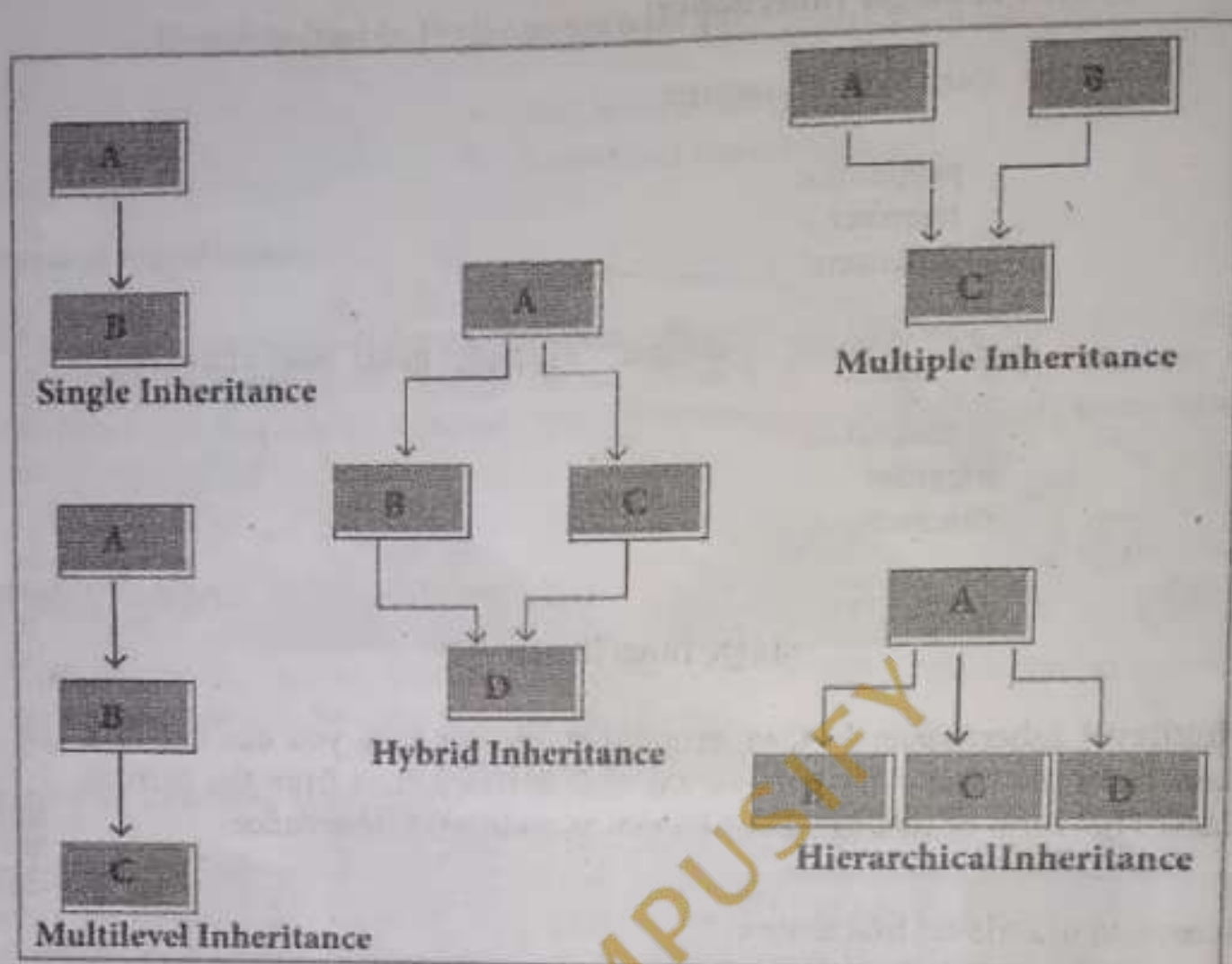
Single Inheritance

Multilevel Inheritance: In C++ programming, not only you can derive a class from the base class but you can also derive a class from the derived class. This form of inheritance is known as multilevel inheritance.

Syntax of multilevel Inheritance:

```
class base_classname
{
    properties;
    member
    functions;
};
class derived_classname1 : visibility_mode base_classname
{
    properties;
    member
    functions;
};
class derived_classname2 : visibility_mode derived_classname1
{
    properties;
    member
    functions;
};
```

Pictorial Representation:



VII Resources Required

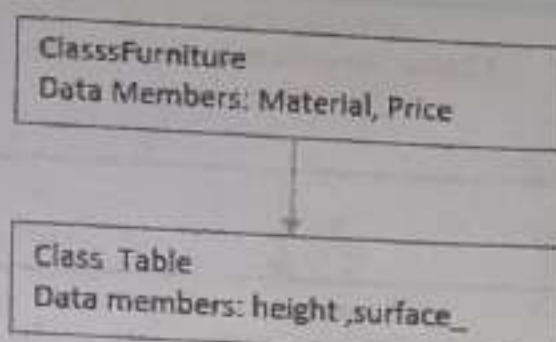
Sr. No	Name Resource	Specification	Quantity	Remarks
1	Computer System	Any desktop or laptop computer with basic configuration	One computer system for each student	
2	Operating system	Windows / LINUX	One for each computer system	
3	Software	Turbo C++ Version 3.0 or any other	One for each computer system	

VIII Precautions to be followed

1. Handle computer system and peripherals with care.
2. Follow safety practices.

IX Exercise:

1. WAP to implement inheritance shown below figure. Assume suitable member function.



2. Write a C++ program to define a class "Employee" having data members emp_no, emp_name and emp_designation. Derive a class "Salary" from "Employee" having data members basic, hra, da, gross_sal. Accept and display data for one employee.

X C++ code:

Write "C++" Code for above exercise on the blank pages attached at the end of practical.

XI Resources Used

Sr. No.	Name of Resource	Suggested Broad Specification	Quantity
1.	Computer system	any desktop	1

XII Result(s)

we have executed this program
Single level inheritance

XIII Conclusion

we have successfully single and multilevel inheritance.

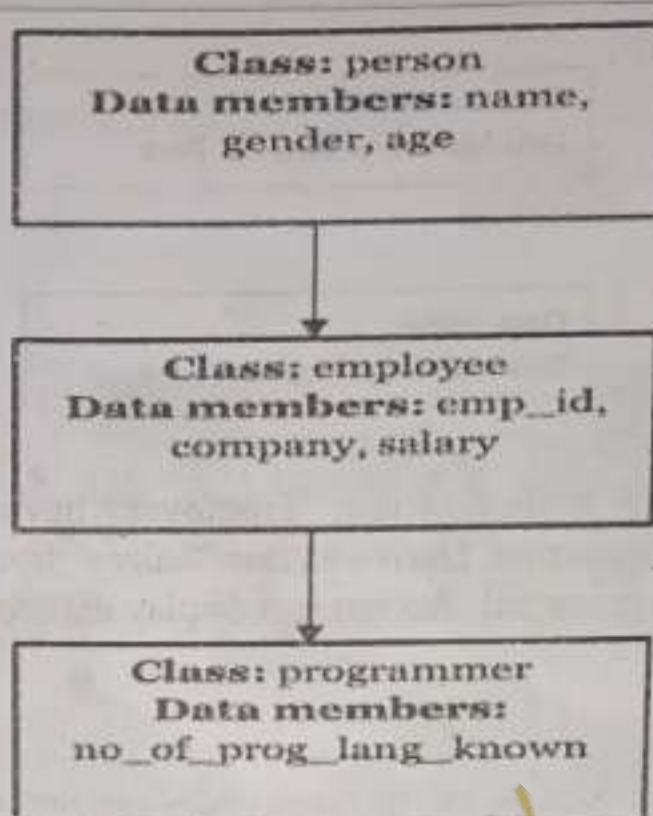
XIV Practical Related Questions

Note: Below given are few sample questions for reference. Teacher must design more such questions so as to ensure the achievement of identified CO.

(Note: for all relevant programming exercise use blank pages provided or attach more pages if needed.)

1. Write a C++ program to define a class "Student" having data members roll_no, name. Derive a class "Marks" from "Student" having data members m1, m2, m3, total and percentage. Accept and display data for one student.

2. Write a C++ program to implement following Multilevel Inheritance.



3. Complete the following table:

Program Code	Write & justify Output
<pre> a) #include<iostream.h> class Base { public: Base(){} ~Base(){} protected: private: }; class Derived:public Base { public: Derived(){} Derived(){} private: protected: }; void main() { cout << "The program exeuted" << endl; } </pre>	The program executed.

b) #include <iostream.h>

class A

```
{  
public:  
void print() {  
cout <<  
"print() in A";  
}  
};
```

class B
private A

```
{  
public:  
void print() {  
cout <<  
"print() in B";  
}  
};
```

class C :

public B

```
{  
public:  
void print() {  
cout <<  
"print() in C";  
A::print();  
};
```

void main()

```
{  
Cb b; b.print();  
}
```

print() in C
print() in A

(Space for Answers)

Exercise :

Q.1]

```
#include <iostream.h>
#include <conio.h>
class furniture
Public:
String material;
double price;
Furniture (String material, double price) {
    this -> material = material;
    this -> price = price;
}

void display FurnitureDetails () {
    cout << "Material : " << material;
    cout << "Price : " << price;
} };

Class Table : Public Furniture {
Public:
double height;
String surface-finish;
Table (String material, double price, double
height, String surface-finish);
Furniture (material, price) {
    this -> height = height;
    this -> surface-finish = surface-finish;
}

void displayTableDetails () {
    displayFurnitureDetails ();
    cout << "Height : " << height << endl;
    cout << "Surface Finish : " << surface-finish << endl;
} };
```



```

int main () {
    Table myTable ("wood", 200.0, 3.0, "Glossy");
    myTable.displayTableDetails();
    return 0;
}

```

Q.2] #include <iostream.h>
#include <string.h>
class Employee {
 public:
 int emp-no;
 string emp-name;
 string emp-designation;
 void getEmployeeData () {
 cout << "Enter Employee Number:";
 cin >> emp-no;
 cout << "Enter Employee Name:";
 cin.ignore();
 getline (cin, emp-name);
 cout << "Enter Employee Designation:";
 cin >> emp-designation;
 }
 void displayEmployeeData () {
 cout << "\n Employee details:\n";
 cout << "Employee Number : " << emp-no << endl;
 cout << "Employee Name : " << emp-name;
 cout << "Employee Designation : " << emp-designation;
 }
};
class salary : public Employee {
 public:

```

double basic, hra, da, gross-sal;
void getSalaryData() {
    cout << "Enter Basic salary: ";
    cin >> basic;
    cout << "Enter HRA: ";
    cin >> hra;
    cout << "Enter DA: ";
    cin >> da;
}

void calculateGrossSalary() {
    gross-sal = basic + hra + da;
}

void displaySalary() {
    gross-sal = basic + hra + da;
}

void displaySalaryData() {
    cout << "Salary Details: \n";
    cout << "Basic Salary: " << basic << endl;
    cout << "HRA: " << hra << endl;
    cout << "DA: " << da << endl;
    cout << "Gross salary: " << gross-sal << endl;
}

int main() {
    Salary employee;
    employee.getEmployeeData();
    employee.getSalaryData();
    employee.calculateGrossSalary();
    employee.displayEmployeeData();
    employee.displaySalaryData();
    return 0;
}

```


Practical Related Questions

Q.1] #include <iostream.h>

#include <string>

class student {

public:

int roll-no;

string name;

void getstudentData();

cout << "Enter Roll number:";

cin >> roll-no;

cout << "Enter student name:";

cin.ignore();

getline(cin, name);

}

void displaystudentData();

cout << "\n studentDetails: \n";

cout << "Roll Number:" << roll-no << endl;

cout << "student Name:" << name << endl;

{ }

class marks : public student {

public:

int m1, m2, m3, total;

double percentage;

void getmarksData();

cout << "\n Enter marks for subject 1:";

cin >> m1;

cout << "Enter marks for subject 2:";

cin >> m2;

cout << "Enter marks for subject 3:";

cin >> m3;

}

```
cout << "Beep beep!" << endl;
```

```
} };
```

```
class SportsCar : public Car {
```

```
public:
```

```
    void race();
```

```
    cout << "vroom vroom!" << endl;
```

```
} };
```

```
int main() {
```

```
    SportsCar mySportsCar;
```

```
    mySportsCar.move();
```

```
    mySportsCar.honk();
```

```
    mySportsCar.race();
```

```
    return 0;
```

```
}
```



CAMPUSIFY

Practical No.12: Write programs to implement Multiple Inheritance

I Practical Significance:

Multiple Inheritance is a feature of C++ where a class can inherit from more than one classes.

II Industry/Employer Expected Outcome(s)

Multiple inheritance allows you to mix properties and behaviors from different classes.

III Course Level Learning Outcome(s)

Implement Multiple Inheritance in C++ program.

IV Laboratory Learning Outcome(s)

Write/Compile/debug/Execute simple C++ program using Multiple inheritance.

V Relevant Affective Domain related outcome(s)

1. Select proper programming environment in C++.
2. Follow ethical practices.

VI Relevant Theoretical Background

1. **Inheritance** :- It is the process of inheriting properties of objects of one class by objects of another class. The class which inherits the properties of another class is called Derived or Child or Sub class and The class whose properties are inherited is called Base or Parent or Super class.

2. **Multiple Inheritance** :- When a class is derived from two or more base classes, such inheritance is called Multiple Inheritance. It allow us to combine the features of several existing classes into a single class.

Syntax of Multiple Inheritance

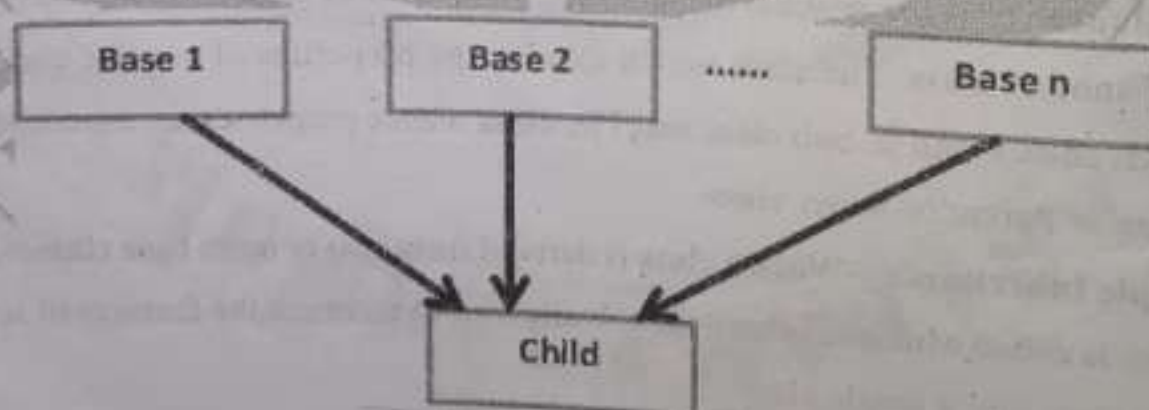
```
class base class1
{
    properties;
    member
    functions;
};
```

```
class base class2
{
    properties;
    member
    functions;
};
```

```
class base classN
{
    properties;
    member
    functions;
};
```

```
class derived_classname : visibility_mode base_class1,
visibility_mode base_class2,...,visibility_mode base_classN
```

```
{
    properties;
    member
    functions;
};
```



The different access specifiers for inheritance concept-

Access	public	protected	Private
Same class	Yes	Yes	Yes
Derived classes	Yes	Yes	No
Outside classes	Yes	No	No

Virtual Base Class:

1. Virtual Base class: An ambiguity can arise when several paths exist to a class from the same base class. This means that a child class could have duplicate sets of members inherited from a single base class.

C++ solves this issue by introducing a virtual base class. When a class is made virtual, necessary care is taken so that the duplication is avoided regardless of the number of paths that exist to the child class.

When two or more objects are derived from a common base class, we can prevent multiple copies of the base class being present in an object derived from those objects by declaring the base class as virtual when it is being inherited. Such a base class is known as virtual base class. This can be achieved by preceding the base class's name with the word **virtual**.

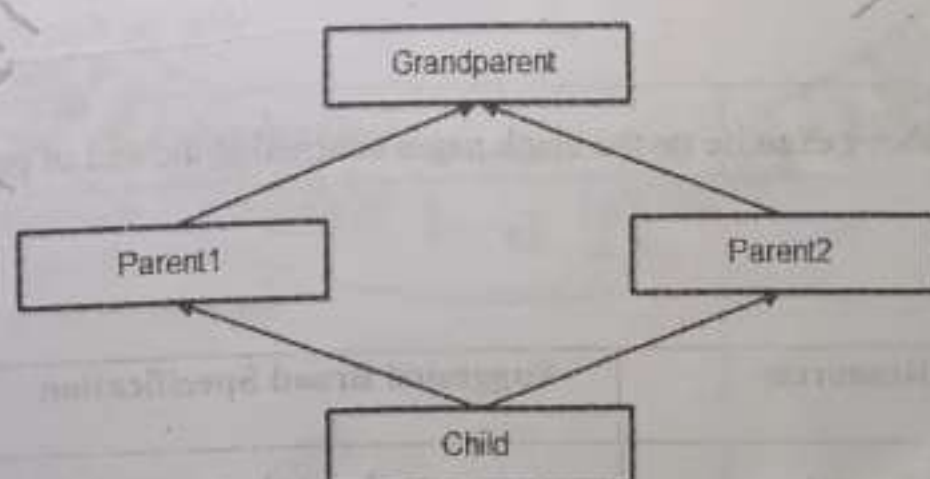
Syntax:

```
class derived_class_name: virtual visibility_mode base_class
```

```
{
```

```
    //members of derived class
```

```
};
```



VII Resources Required

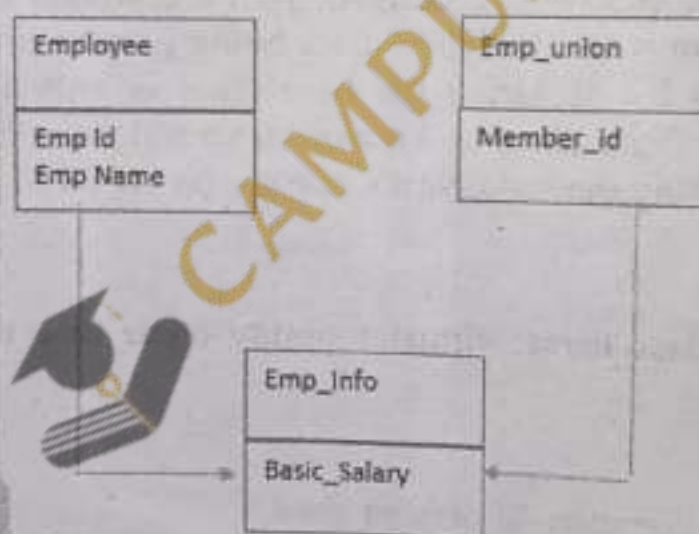
Sr. No	Name Resource	Specification	Quantity	Remarks
1	Computer System	Any desktop or laptop computer with basic configuration	One computer system for each student	
2	Operating system	Windows /LINUX	One for each computer system	
3	Software	Turbo C++ Version 3.0 or any other	One for each computer system	

VIII Precautions to be followed

1. Handle computer system and peripherals with care.
2. Follow safety practices.

IX Exercise:

1. Identify the following type of Inheritance shown . Write a definition of each class .Write suitable member functions to accept and display data for each class.



2. Write a Program to get the average marks of six subjects using the Multiple Inheritance

X C++ code:

Write "C++" Code for above exercise on the blank pages attached at the end of practical.

XI Resources Used

Sr. No.	Name of Resource	Suggested Broad Specification	Quantity
1.	Computer system	any desktop	1

XII Result(s)

we have created multiple inheritance.

XIII Conclusion

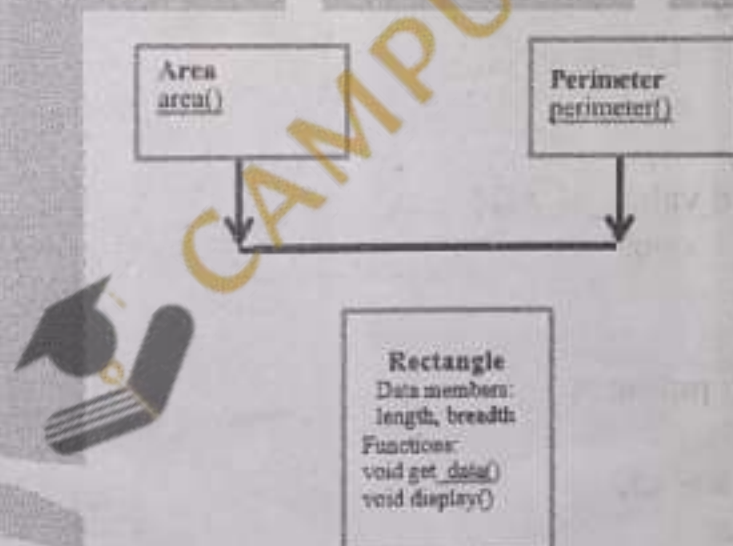
we have successfully multiple inheritance.

XIV Practical Related Questions

Note: Below given are few sample questions for reference. Teacher must design more such questions so as to ensure the achievement of identified CO.

(Note: for all relevant programming exercise use blank pages provided or attach more pages if needed.)

1. Write a C++ program to calculate the area and perimeter of rectangles using concept of inheritance.



2. Complete the following table:

Program Code	Write & justify Output
<pre> a) #include <iostream> #include <string> using namespace std; class A { int a, b; float d; public: void change(int i){ a = i; </pre>	Output :- 8 The size of class B is 8 bytes because it inherits.

```

    }
    void value_of_a(){
    cout<<a;
    }
    };
    class B: private A
    {
    };
    int main(int argc, char const *argv[])
    {
    B b;
    cout<<sizeof(B);
    return 0;
    }

```

```

b) #include <iostream>
#include <string>
using namespace std;
class A
{
    float d;
public:
    int a;
    void change(int i){
        a=i;
    }
    void value_of_a(){
        cout<<a;
    }
};
class B: public A
{
    int a=15;
public:
    void print(){
        cout<<a;
    }
};
int main(int argc, char const *argv[])
{
    B b;
    b.change(10);
    b.print();
    b.value_of_a();

    return 0;
}

```

Output: 1510
 B: a is 15 and
 A: a is 10, as
 shown by the
 calls to print() and value_of_a() respectively.

(Space for Answers)

Exercise

Q.11

```
class employee {
private:
    int emp-id;
    string emp-name;
public:
    employee (int id=0, string name=""): emp-id(id),
    emp-name(name) {}
    void accept data () {
        cout << "Enter employee ID:";
        cin >> emp-id;
        cout << "Enter employee Name:";
        cin.ignore();
        getline (cin, emp-name);
    }
    void displaydata () const {
        cout << "Employee ID:" << emp-id << endl;
        cout << "Employee Name:" << emp-name << endl;
    }
};

class emp-info : public employee {
private:
    float Basic-salary;
public:
    emp-info (int id=0, string name="", float salary=0.0):
    employee (id, name), Basic-salary (salary) {}
    void accept data () {
        //
    }
    employee::acceptdata ();
    cout << "Enter Basic salary:";
```

```

cin >> Basic_salary;
}
void displaydata () const {
employee::displaydata ();
cout << " Basic Salary " << Basic_salary << endl;
} }

```

Derived class 2

```

class emp-union : public employee {
private:
int member_id;
public:
emp-union (int id=0, name=" ");
void acceptdata () {
employee::acceptdata ();
cout << "enter union member ID ";
cin >> member_id;
}
void displaydata () const {
employee::displaydata ();
cout << "union member ID " << member_id << endl;
}
}

```



```

Q.2] #include <iostream.h>
#include <conio.h>
class student
{
protected:
int student ID;
string Name;
public:
void accept ()
{
cout << "enter ID:";
cin >> student_ID;
cout << "enter name:";
cin >> Name;
}
void display ()
{
cout << "student ID" << student_ID;
cout << "student Name" << Name;
}
};

class I % virtual public student.
{
protected:
float subject 1, subject 2, subject 3;
public:
void accept marks ()
{
cout << "enter subject 1";
cin >> subject 1;

```

```

cout << "enter subject 2" << endl;
cin >> subject2;
cout << "enter subject 3" << endl;
cin >> subject3;
}

void display marks1 () const {
    cout << "mark in subject 1:" << subject1 << endl;
    cout << "mark in subject 2:" << subject2 << endl;
    cout << "mark in subject 3:" << subject3 << endl;
} }

class 2 : virtual public student {
protected:
    float subject 4, subject 5, subject 6;
public:
    void accept marks2 () {
        cout << "enter subject 4" << endl;
        cin >> subject 4;
        cout << "enter subject 5" << endl;
        cin >> subject 5;
    }

    void display mark 2 () const {
        cout << "mark subject 4" << subject4 << endl;
        cout << "mark subject 5" << subject5 << endl;
        cout << "mark subject 6" << subject 6 << endl;
    } }

class Result : public marks1, public marks2 {
public:
    void display result () const {
        float total marks = subject 1 + subject 2 + subject 3 +

```



```

Subject 4 + Subject 5 + Subject 6;
float average marks = totalmarks / 6;
cout << "total marks" << total mark;
cout << "average marks" << average mark;
}
};
int main()
{
    result R;
    R.accept();
    R.accept();
    R.accept mark2();
    R.display();
    R.display mark2();
    getch();
}

```

Practical Related Question

```

Q.1] #include <iostream>
#include <conio.h>
class Area {
protected:
    float length, breadth;
public:
    float area()
    {
        return length * breadth;
    }
};
class perimeter

```

```
{
```

```
protected:
```

```
float length, breadth;
```

```
Public:
```

```
float perimeter()
```

```
{
```

```
return 2 * (length + breadth);
```

```
}
```

```
};
```

```
class Rectangle : public Area, public perimeter {
```

```
public:
```

```
void get()
```

```
{
```

```
cout << "length rectangle";
```

```
cin >> length;
```

```
cout << "breadth rectangle";
```

```
cin >> breadth;
```

```
}
```

```
void display()
```

```
{
```

```
cout << "length" << length << "breadth" << breadth;
```

```
cout << "Area of rectangle" << area;
```

```
cout << "perimeter of rectangle" << perimeter();
```

```
}
```

```
};
```

```
void main() {
```

```
Rectangle R;
```

```
R.get();
```

```
R.display();
```

```
getch();
```

```
}
```


Practical No.13: Write programs to implement Hierarchical Inheritance

I Practical Significance:

Hierarchical Inheritance is a type of inheritance in C++ where one base class is inherited by more than one derived class.

II Industry/Employer Expected Outcome(s)

Hierarchical Inheritance allows for code reuse and refactoring, as the same code can be reused in multiple classes.

III Course Level Learning Outcome(s)

Implement Hierarchical Inheritance in C++ program.

IV Laboratory Learning Outcome(s)

Write/ Compile/ debug / Execute simple C++ program using Hierarchical Inheritance.

V Relevant Affective Domain related outcome(s)

- Select proper programming environment in C++.
- Follow ethical practices.

VI Relevant Theoretical Background

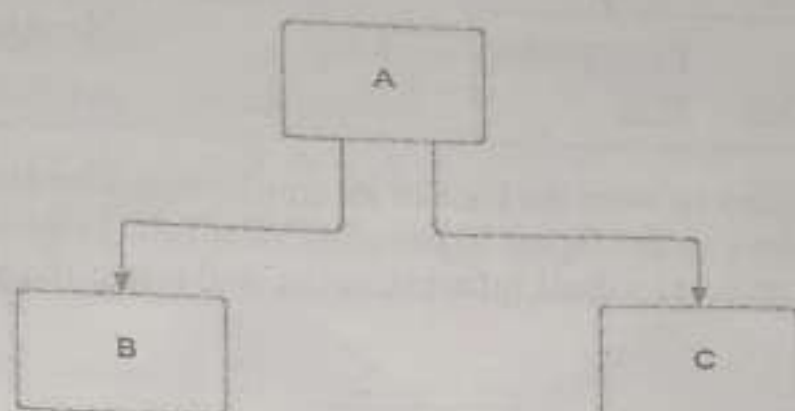
- Inheritance** :-It is the process of inheriting properties of objects of one class by objects of another class. The class which inherits the properties of another class is called Derived or Child or Sub class and The class whose properties are inherited is called Base or Parent or Super class.
- Hierarchical Inheritance** -It is feature of Object-Oriented-programming in which a derived class (child class) inherits the property (data member and member functions) of the Base class (parent class). For example, a child inherits the traits of their parents.

Syntax of Multiple Inheritance

```
Class A
{
    .....
};
Class B: access_specifier A
{
```

```

.....
};
Class C: access_specifier A
{
.....
};
    
```



Visibility Modes in Hierarchical Inheritance in C++ :

The visibility modes in hierarchical inheritance in C++ are Public, Protected, and Private. This means that the base class members (which includes functions and data variables) can be inherited as public, protected, or private according to the user's needs.

Public: Base class members declared public will be inherited as public members in derived classes and can be accessed by both the derived class and any external user.

Protected: Base class members declared protected will only be accessible within the derived classes(of that base class) unless it is explicitly declared public within the derived classes.

Private: If we derive a subclass from a Private base class. Then both public member and protected members of the base class will become Private in the derived class.

VII Resources Required



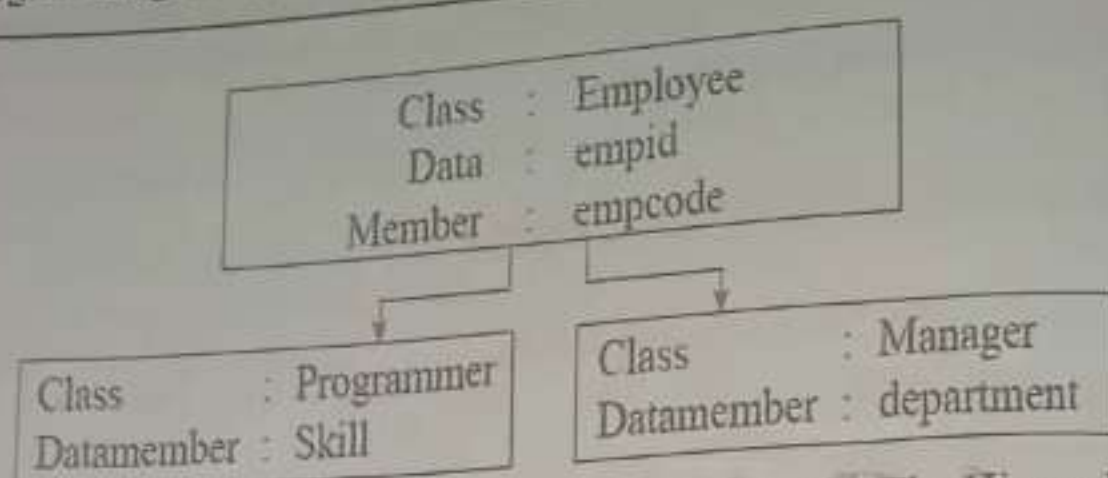
Sr. No	Name Resource	Specification	Quantity	Remarks
1	Computer System	Any desktop or laptop computer with basic configuration	One computer system for each student	
2	Operating system	Windows /LINUX	One for each computer system	
3	Software	Turbo C++ Version 3.0 or any other	One for each computer system	

VIII Precautions to be followed

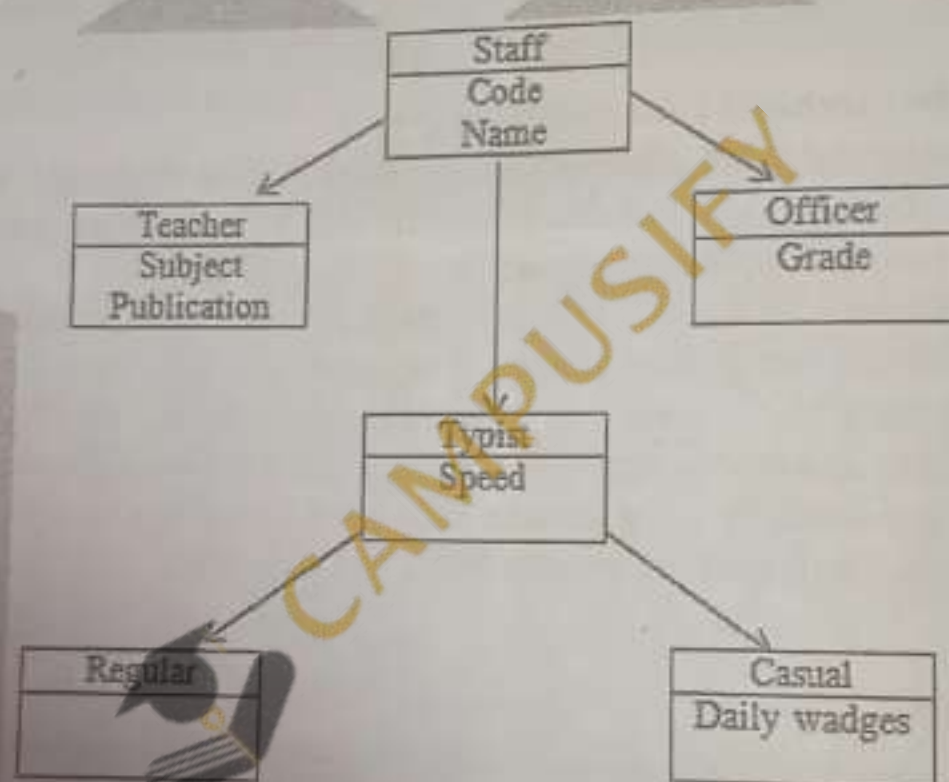
1. Handle computer system and peripherals with care.
2. Follow safety practices.

IX Exercise:

1. Write a C++ program to implement the following inheritance. Accept and display data for one programmer and one manager. Make display function virtual.



2. An Educational institute wishes to store the data of its employees. The Hierarchical relationship of its related classes are shown in the figure below. Define all the classes to represent hierarchy and define functions to retrieve individual information as and when required.



X C++ code:

Write "C++" Code for above exercise on the blank pages attached at the end of practical.

XI Resources Used

Sr. No.	Name of Resource	Suggested Broad Specification	Quantity
1.	Computer System	Any desktop	1
2.	Operating System	windows	1
3.	Software	Turbo c++	1

XII Result(s)

we have executed this program Hierarchical inheritance

XIII Conclusion

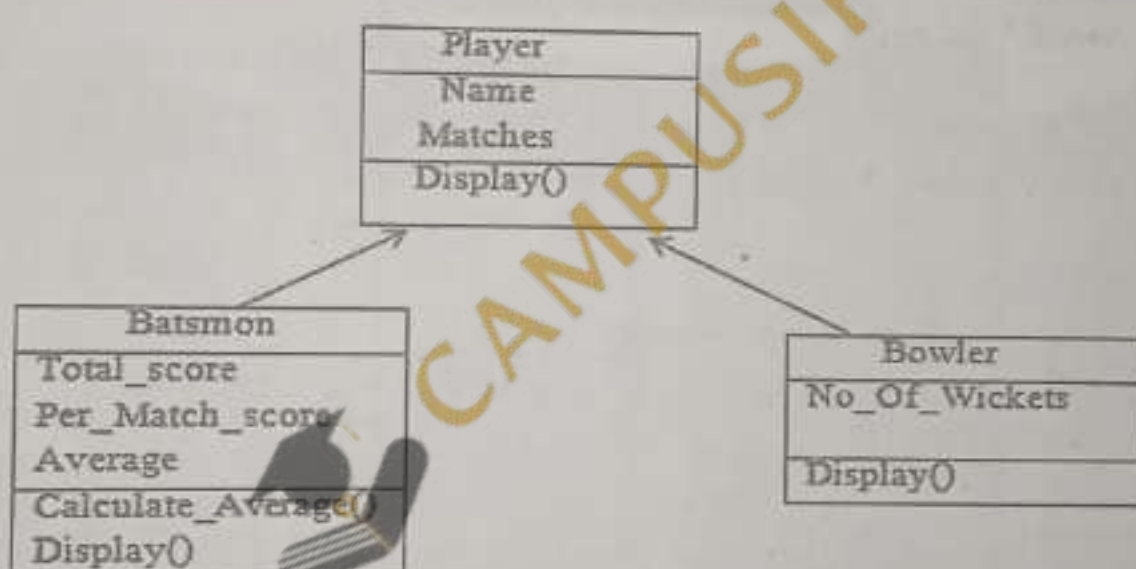
we have successfully class hierarchies promoting code modularity

XIV Practical Related Questions

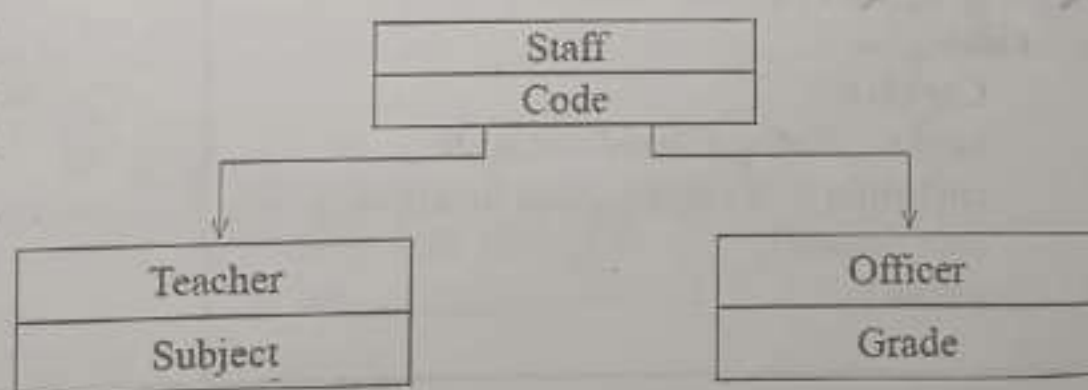
Note: Below given are few sample questions for reference. Teacher must design more such questions so as to ensure the achievement of identified CO.

(Note: for all relevant programming exercise use blank pages provided or attach more pages if needed.)

1. Write a C++ program to implement given class hierarchy.



2. Write a C++ program to implement given figure class hierarchy.



3. Complete the following table:

Program Code	Write & justify Output
<pre> a) #include <iostream> using namespace std; class Animal { public: void info() { cout << "I am an animal." << endl; } }; class Dog : public Animal { public: void bark() { cout << "I am a Dog. Woof woof." << endl; } }; class Cat : public Animal { public: void meow() { cout << "I am a Cat. Meow." << endl; } }; int main() { class Dog dog1; cout << "Dog Class:" << endl; dog1.info(); // parent Class function dog1.bark(); class Cat cat1; cout << "\nCat Class:" << endl; cat1.info(); // parent Class function cat1.meow(); return 0; } </pre>	Dog class: I am an animal. I am Dog. woof woof. Cat class: I am an animal I am an cat meow.

```

b)#include<iostream.h>
#include<conio.h>
class emp
{
int id;
char name[20];
int sal;
public:
void accept()
{
cout<<"enter id, name and salary"<<endl;
cin>>id>>name>>sal;
}
void dis()
{
cout<<"id"<<id;
cout<<"name--"<<name<<endl;
cout<<"salary"<<sal;
}
};
class manager : public emp
{
public:
acc1()
{
accept();
dis();
}
};
class clerk : public emp
{
public: acc2()
{
accept();
dis();
}
};
void main()
{
manager m;
clerk c;
m.acc1();
c.acc2();
getch();
}

```

enter id, name and
Salary 101 Ravf
20000 id 101 name -
Ravf Salary 201
Raj 15000 id 201
name - Raj salary
15000

(Space for Answers)

Practical Related Question.

Q.17 #include <iostream.h>

#include <conio.h>

class player

{

public:

String name;

int matches;

player (String n, int m)

{

name = n;

mathe = m;

?

virtual void display()

{

cout << "Name " << name << endl;

cout << "Matches " << matches << endl;

};

class Batsman : public player

{

public:

int total-score;

double per-match-score;

double average;

Batsman (String n, int m, int ts) : player (n, m)

{

total-score = ts;

per-match-score

average = per-match-score;

?

```

void calculate_average()
{
    average = (double) total_scores;
}

void display() override
{
    player::display();
    cout << "total score" << total_score << endl;
    cout << "permatch score" << per_match_score << endl;
    cout << "Average " << average << endl;
}

};

class Bowler : public player
{
public:
    int no_of_wickets;
    Bowler(string n, int m, int now) : player(m, n)
    {
        no_of_wickets = now;
    }

    void display() override
    {
        player::display();
    }

    cout << "No- of wickets " << no_of_wickets;
}

};

int main()
{
    string name;

```



```

int matches, total-score, no-of-wickets;
cout << "enter details for batsman";
cout << "name";
getline(cin, name);
cout << "matches";
cin >> matches;
cout << "Total score";
cin >> total-score;
Batsman batsman (name, matches, total-score);
cout << "enter details for bowler";
cout << "name";
cout << "Matches";
cin >> matches;
cout << "No-of-wickets";
cin >> no-of-wickets;
Bowler bowler (name, matches, no-of-wickets);
cout << "Batsman details";
batsman display();
getch();
}

```

Q.2] #include <iostream.h>

#include <conio.h>

class staff

{

public:

int code;

staff (int c)

{

code = c;

}

Virtual void display ()

{

cout << "staff code" << code << endl;

};

class Teacher : public staff {

public:

String subject;

Teacher (int c, String s): staff (c)

{

~~Teacher~~ subject = s;

}

void display () override

{

Staff : display ();

cout << "subject" << subject << endl;

}

};

class officer : public staff

{

public:

int grade;

officer (int c, int g): staff (c)

{

grade = g;

}

void display () override

{

Staff :: display ();

cout << "Grade " << grade << endl;

};


```
String skill;
```

```
programmer (int id, int code, String s)
```

```
{
```

```
skill = s;
```

```
}
```

```
void display () override {
```

```
employee :: display ();
```

```
cout << "Skill" << skill << endl;
```

```
}
```

```
};
```

```
class manager : public employee
```

```
{
```

```
public:
```

```
String department;
```

```
manager (int id, int code, String d) : employee
```

```
(id, code) {
```

```
department = d;
```

```
}
```

```
void display () override {
```

```
employee :: display ();
```

```
cout << "Department" << department << endl;
```

```
};
```

```
void main () {
```

```
int empid, empcode;
```

```
String skill, department;
```

```
cout << "Enter details for programmer";
```

```
cout << "Employee ID";
```

```
cin >> empid;
```

```
cout << "Employee Code";
```

```
cin >> empcode;
```

```

int main()
{
    Teacher t (101, "math");
    officer o (201, 3);
    t.display();
    cout << " " << endl;
    o.display();
    return 0;
}

```

Exercise

```

Q.11 #include <iostream.h>
#include <conio.h>
class employee {
public:
    int empid;
    int empcode;
    employee (int id, int code)
    {
        empid = id;
        empcode = code;
    }
    virtual void display ()
    {
        cout << "employee id" << empid << endl;
        cout << "employee code" << empcode << endl;
    }
};
class programmer : public employee {
public:

```



```

String publication;
Teacher (int c, string n, string s, string p):
    staff (c, n) {
    Subject = s;
    Publication = p;
}

void display () override
{
    staff::display();
    cout << "subject" << subject << endl;
    cout << "publication" << publication << endl;
}

class officer : public staff
{
    public:
    int grade;
    officer (int c, string n, int g): staff (c, n)
    {
        grade = g;
    }

    void display () override
    {
        staff::display();
        cout << "Grade" << grade << endl;
    }

    class typist : public staff
    {
        public:
        int speed;
        Typist (int c, string n, int s): staff (c, n)
    }
}

```

```

cout << "Department";
cin >> department;
manager manager (empid, empcode, department);
cout << "manager details";
programmer, display ();
cout << "manager details";
manager, display ();
getch();
}

```

```

Q.27 #include <iostream.h>
#include <string.h>
class Staff {
public:
    int code;
    string name;
    Staff (int c, string n)
    {
        code = c;
        name = n;
    }

```

```

    virtual void display ()
    {
        cout << "staff code " << code << endl;
        cout << "staff name " << name << endl;
    }
};

```

```

class Teacher : public Staff
{
public:
    string subject;

```


I
{ Speed = 5;
}

void display() override {
Staff :: display();

II
cout << "Speed" << speed << endl;
};

III
class Regular : public Typist {
public:

IV
Regular (int c, string n, int s) : Typist (n, s)
{
};

V
class Casual : public Typist {
public:

double dailywages;

Causal (int c, int s, double dw) : Typist (c, s)

VI
{
dailywages = dw;

void display() override

{
Typist :: display();

cout << "Daily wage" << dailywages << endl;

}

};

int main() {

Teacher t (101, "John Doe", "Math", "ABC publication");

Officer o (201, "Jane Smith", 3);

Regular r (301, "Alice Johnson", 60);

Causal c (401, "Bob Lee", 55, 500.0);

t.display();

```
cout << endl;
```

```
display();
```

```
cout << endl;
```

```
R. display();
```

```
cout << endl;
```

```
causal . display();
```

```
getch();
```

```
}
```



CAMPUSIFY

Syntax:
 class derived_class_name: virtual visibility_mode base_class
 {

-----//members of derived class
 };

Example:

```
class A
{
    Public:
    Int i;
};

class B: virtual public A
{
    Public:
    Int j;
};
```

VII Resources Required

Sr. No	Name Resource	Specification	Quantity	Remarks
1	Computer System	Any desktop or laptop computer with basic configuration	One computer system for each student	
2	Operating system	Windows /LINUX	One for each computer system	
3	Software	Turbo C++ Version 3.0 or any other	One for each computer system	

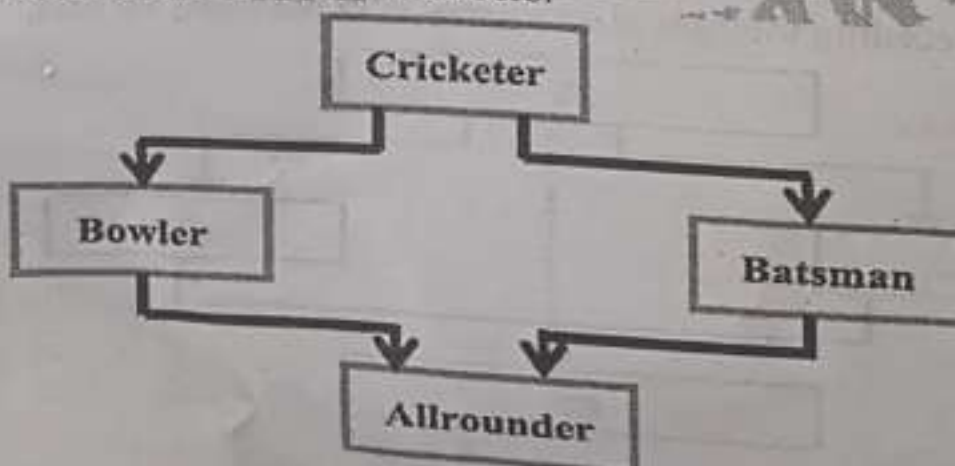
VIII Precautions to be followed

1. Handle computer system and peripherals with care.
2. Follow safety practices.

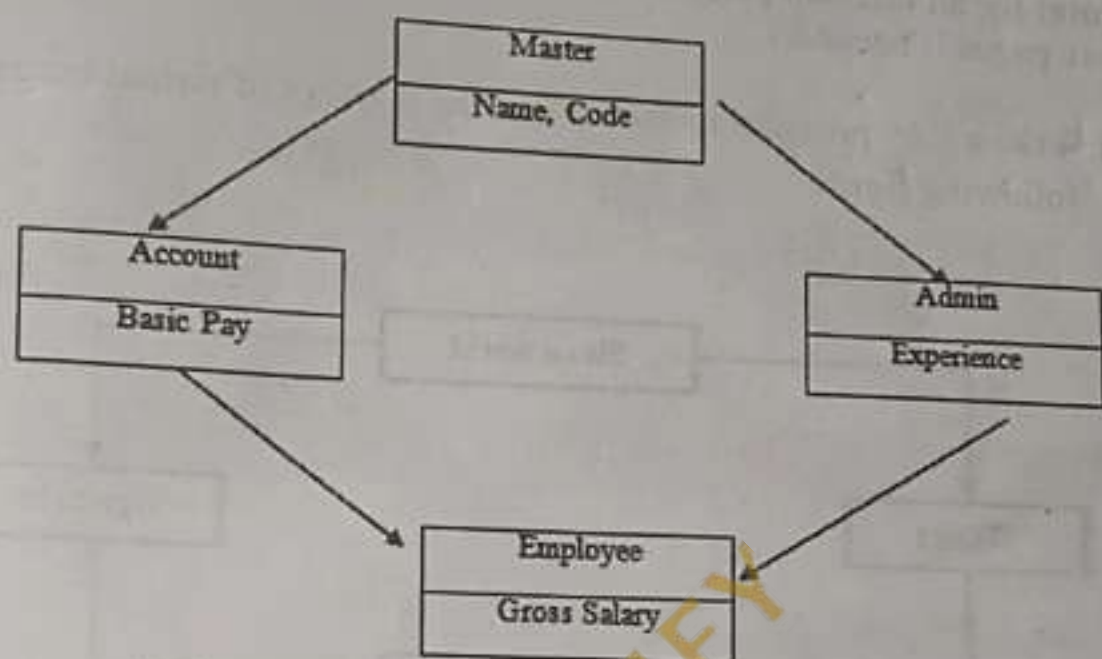
IX Exercise:

1. Write a C++ program to implement the concept of Virtual Base Class for following figure.

Assume suitable data and function members.



2. Write a C++ program to implement the concept of Virtual Base Class for following figure.
Accept and Display information of one employee with his name,code,basic Pay,Experience and Gross Salary with the object of Employee class.



X C++ code:

Write "C++" Code for above exercise on the blank pages attached at the end of practical.

XI Resources Used

Sr. No.	Name of Resource	Suggested Broad Specification	Quantity
1.	Computer system	Any desktop	1

XII Result(s)

.....we have executed this program virtual.....
.....base class.....

XIII Conclusion

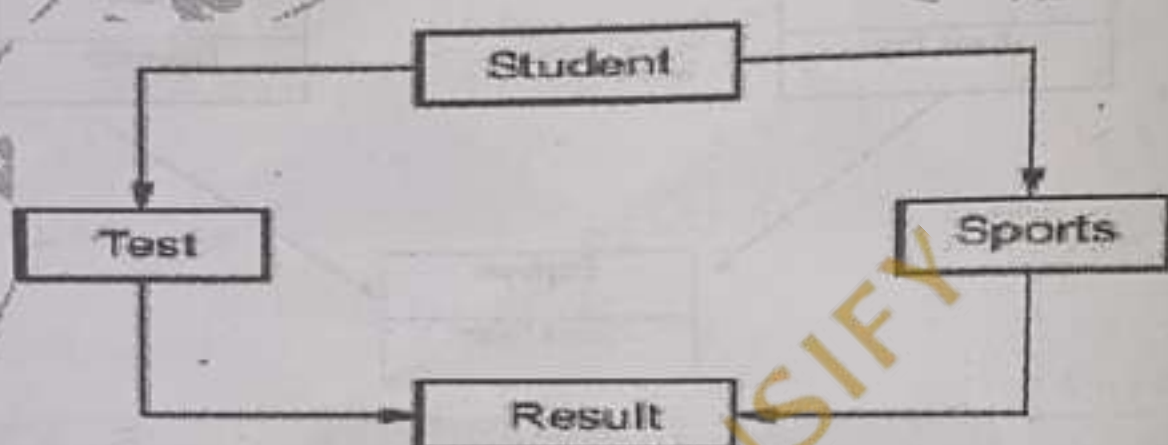
.....we successfully completed oop.....
.....in writing c++ programs to implement Virtual Base class.....

XIV Practical Related Questions

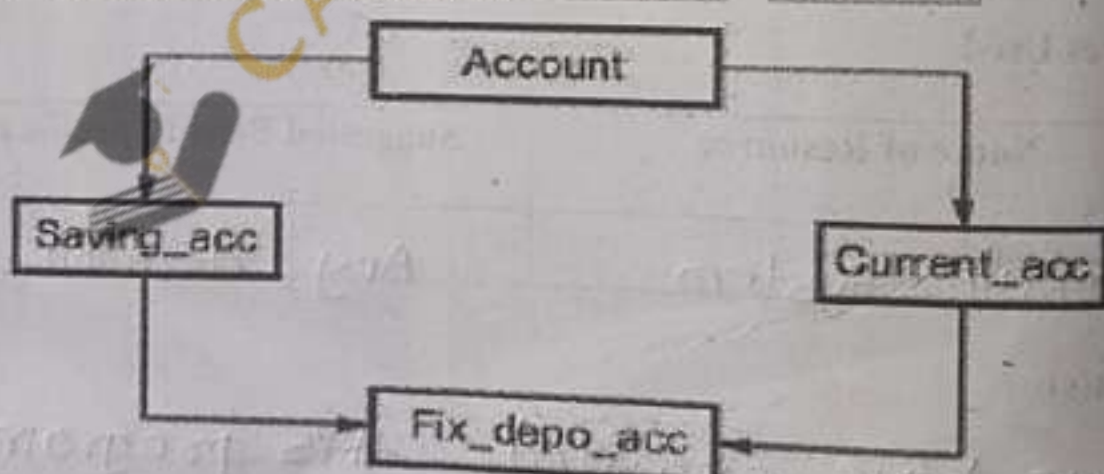
Note: Below given are few sample questions for reference. Teacher must design more such questions so as to ensure the achievement of identified CO.

(Note: for all relevant programming exercise use blank pages provided or attach more pages if needed.)

1. Write a C++ program to implement the concept of virtual base class for following figure.



2. Write a C++ program to implement given figure class hierarchy. Assume suitable member variables and member functions.



XIII Complete the following table:

Program Code	Write & justify Output
<pre> a) #include <iostream.h> struct a { int count; }; Struct b </pre>	

```
{  
    int* value;  
};  
struct c  
public a,  
public b  
};  
int main()  
{  
    c* p new =c;  
    p->value=0;  
    cout<<"Inherited";  
    return 0;  
}
```

```
b) #include <iostream.h>  
class A  
{  
public:  
    void display()  
    {  
        cout<<"A\n";  
    }  
};  
class B : public A  
{  
public:  
    void display()  
    {  
        cout<<"B\n";  
    }  
};  
int main()  
{  
    B b;  
    b.display();  
    b.A::display();  
    b.B::display();  
    return 0;  
}
```

Output :-
A

(Space for Answers)

Practical Related Question

```
Q.1] #include <iostream.h>
#include <string.h>
#include <conio.h>
class student {
public:
    int rollno;
    string name;
    student (int r, string n): rollno(r), name(n) {}
    void display student () {
        cout << "Roll no" << rollno << endl;
        cout << "Name" << name;
    }
};
class Test : virtual public student
{
public:
    float testmarks;
    Test (int r, string n, float marks): student(r,n),
    test Marks (marks) {}
    void displaytest ()
    {
        cout << "test mark" << testmarks << endl;
    }
};
class Sports : virtual public student
{
    float Sportscore;
    Sport (int r, string n, float score): student(r,n) {
        void displaysports () {
            cout << "sports score" << sportscore << endl;
        }
    }
};
```

```
class Result : public test, public sport {
public:
    float total;
    result (int r, String g, float marks, float score)
: Student (r, n), test (r, n, mark) {
    total = test marks + Sports score;
}

    void display () {
        display student ();
        display test ();
        display sports ();
        cout << "total score " << total << endl;
    }

    void main () {
        int roll no;
        String names;
        float testmarks;
        cout << "enter Roll no ";
        cin >> roll no;
        cout << "enter name ";
        cin >> "name ";
        result . display result ();
        getch ();
    }
}
```



```
0.2] #include <iostream.h>
```

```
#include <conio.h>
```

```
class Account
```

```
{
```

```
protected:
```

```
double balance;
```

```
public:
```

```
Account (double bal): balance (bal) {}
```

```
virtual void displayBalance() {
```

```
cout << "Account Balance " << balance << endl;
```

```
}
```

```
void deposit (double amount) {
```

```
balance += amount;
```

```
cout << "Deposit " << amount << "New Balance" << endl;
```

```
}
```

```
virtual void withdraw (double amount) = 0;
```

```
};
```

```
class Saving_acc : public Account {
```

```
public:
```

```
Saving_acc (double bal): Account (bal) {}
```

```
void withdraw (double amount) override {
```

```
if (balance >= amount) {
```

```
balance -= amount;
```

```
cout << "Withdraw " << amount << "New Balance" <<
```

```
balance << endl; }
```

```
else {
```

```
cout << "Insufficient balance " << endl;
```

```
}}
```

```
void displayBalance() override {
```

```
cout << "Saving Account Balance " << balance << endl;
```

```
};
```

```
Current . display Balance ();  
fixed - deposit . display Balance ();  
Saving . deposit (200.0);  
Saving . withdraw (150.0);  
Current . deposit (500.0);  
Saving . display balance ();  
Current . display balance ();  
Fixed . deposit - display Balance ();  
getch ();  
}
```

Exercise

```
Q.1] #include <iostream.h>  
#include <conio.h>  
class Cricketer {  
public:  
    String name;  
    int age;  
    Cricketer (String n, int a): name(n),  
        age (a) {}  
    void display ()  
    {  
        cout << "Name" << name << endl;  
        cout << "Age" << age << endl;  
    }  
};  
class Bowler : virtual public Cricketer  
{  
public:  
    int wickets;  
    Bowler (String n, int a, int w): Cricketer (n, a)
```



```
wicket s(w) {}
```

```
void display ()
```

```
{
```

```
Cricketter::display ();
```

```
cout << "wicket taken " << wicket s << endl;
```

```
};
```

```
class Batsman : virtual public Cricketter
```

```
{
```

```
public:
```

```
int runs;
```

```
Batsman (string n, int a, int r): Cricketter (n, a,
```

```
runs (r)) {}
```

```
void display ()
```

```
{
```

```
Cricketter::display ();
```

```
cout << "Runs scored " << runs << endl;
```

```
};
```

```
void main () {
```

```
Cricketter::display ();
```

```
cout << "wickets taken " << Bowler::wickets << endl;
```

```
};
```

```
void main ()
```

```
{
```

```
Allrounder player ("John Doe", 30, 150, 2000);
```

```
player.display ();
```

```
getch ();
```

```
}
```

```
cout << "experience" << experience << "years" << endl;
??;
```

```
class employee : public Account, public Admin {
public:
```

```
float grossSalary;
```

```
employee (String n, float bp, int exp)
```

```
: master (n, c) Account (n, c, bp) Admin (n, c, exp) {
```

```
grossSalary = basic pay * (basic pay * 0.2) +
(experience * 100);
```

```
}
```

```
void display employee () {
```

```
display master ();
```

```
display Account ();
```

```
display Admin ();
```

```
cout << "Gross Salary" << grossSalary << endl;
```

```
??;
```

```
void main () {
```

```
String name;
```

```
int Code, experience;
```

```
float basic pay;
```

```
cout << "enter code";
```

```
cin >> code;
```

```
cout << "enter basic pay";
```

```
cin >> basic pay;
```

```
cout << "enter experience (in years);
```

```
cin >> experience;
```

```
employee emp (name, code, basic pay, experi-
ence);
```

```
cout << "employee information";
```

```
emp.display employee ();
```

```
getch ();
```

```
}
```


Practical No.15: Write programs which show use of constructors in derived class.

I Practical Significance:

Constructors in derived classes are special member functions used to initialize objects of derived classes. When you create an object of a derived class, its constructor is called to initialize both the base class part and the derived class part of the object

II Industry/Employer Expected Outcome(s)

When a derived class is extended from the base class, the constructor of the base class is executed first followed by the constructor of the derived class.

III Course Level Learning Outcome(s)

Implement constructors in derived class in C++ program.

IV Laboratory Learning Outcome(s)

Write/ Compile/ debug /Execute simple C++ program using Constructors in derived class.

V Relevant Affective Domain related outcome(s)

- c. Select proper programming environment in C++.
- d. Follow ethical practices.

VI Relevant Theoretical Background

Constructors in Derived Class in C++

- We can use constructors in derived classes in C++
- If the base class constructor does not have any arguments, there is no need for any constructor in the derived class
- But if there are one or more arguments in the base class constructor, derived class need to pass argument to the base class constructor
- If both base and derived classes have constructors, base class constructor is executed first

Syntax:

```
Derived-Constructor (arg1, arg2, arg3....); Base 1-Constructor (arg1,arg2),  
Base 2-Constructor(arg3,arg4)  
{  
....  
} Base 1-Constructor (arg1,arg2)
```

```

"<<data1<<endl;
    }
};

class Base2{
    int data2;

public:
    Base2(int i){
        data2 = i;
        cout << "Base2 class constructor
called" << endl;
    }
    void printDataBase2(void){
        cout << "The value of data2 is " <<
data2 << endl;
    }
};

class Derived: public Base2, public Base1{
    int derived1, derived2;
public:
    Derived(int a, int b, int c, int d) :
Base2(b), Base1(a, c) {
        derived1 = c;
        derived2 = d;
        cout << "Derived class constructor
called" << endl;
    }
    void printDataDerived(void)
    {
        cout << "The value of derived1 is " <<
derived1 << endl;
        cout << "The value of derived2 is " <<
derived2 << endl;
    }
};

```

The value of
data 1 is: 1

The value of data
2 is: 2

The value of
derived 1 is: 3

The value of
derived 2 is: 4

b) #include <iostream>
c) using namespace std;

```

class A
{
public:
    A(int n )
    {
        cout << n;
    }
};

```

54.3R


```
class B: public A
{
    public:
    B(int n, double d)
    : A(n)
    {
        cout << d;
    }
};
class C: public B
{
    public:
    C(int n, double d, char ch)
    : B(n, d)
    {
        cout << ch;
    }
};
int main()
{
    C c(5, 4.3, 'R');
    return 0;
}
```

(Space for Answers)

Exercise:-

Q. 1] Ans:-

Inside base class
Inside sub class

Q. 2] Ans:-

Inside first base class
Inside second base class
Inside child class.

```
};  
class C: public B
```

• Practical Related question

1] #include <iostream.h>

#include <conio.h>

class Base {

Public:

Base (int x) {

data = x;

cout << "Base constructor called with x=" << x;

}

Protected:

int data;

};

class derived : public Base {

Public:

Derived (int x, int y) : Base (x) {

data = y;

cout << "Derived constructor called with y" << y;

}

Private:

int data 2;

};

void main()

{

Derived obj (10, 20)

~~return~~ & getch();

}



CAMPUSIFY

Practical No.16: Write programs to implement-

- Pointer to object
- 'this' pointer

I Practical Significance:

The pointer to objects helps to improve the efficiency of the program and access the objects dynamically.

II Industry/Employer Expected Outcome(s)

This practical is expected to develop the following skills as:

- i. Define real life problems using pointer to objects.
- ii. Use of this pointer

III Course Level Learning Outcome(s)

Implement Polymorphism in C++.

IV Laboratory Learning Outcome(s)

- Implement pointer arithmetic in a program.
- Implement pointer to object in a program.
- Implement 'this' pointer in a program.

V Relevant Affective Domain related outcome(s)

1. Select proper programming environment in C++.
2. Follow ethical practices.

VI Relevant Theoretical Background

Pointers to objects: -

A pointer is a variable that stores the memory address of another variable (or object) as its value. A pointer aims to point to a data type which may be int, character, double, etc.

Pointers to objects aim to make a pointer that can access the object, not the variables. Pointer to object in C++ refers to accessing an object.

There are two approaches by which you can access an object. One is directly and the other is by using a pointer to an object in C++.

A pointer to an object in C++ is used to store the address of an object. For creating a pointer to an object in C++, we use the following syntax:

Class name *pointer to object;

For storing the address of an object into a pointer in c++, we use the following syntax:

Pointer to object=&object- name;

The above syntax can be used to store the address in the pointer to the object. After storing the address in the pointer to the object, the member function can be called using the pointer to the object with the help of an arrow operator.

Examples

Example 1. In the below example, a simple class named My_Class is created. An object of the class is defined as named object. Here a pointer is also defined named p. In the program given below program, it is shown how can we access the object directly and how can we use the pointer to access the object directly.

```
// Example using an object pointer.

#include <iostream>
using namespace std;

class My_Class {
    int num;
    public:
    void set_number(int value)
    {
        num = value;
    }
    void show_number();
};

void My_Class::show_number()
{
    cout << num << "\n";
}

int main()
{
    My_Class object, *p; // an object is declared and a pointer to it

    object.set_number(1); // object is accessed directly
    object.show_number();

    p = &object; // the address of the object is assigned to p
    p->show_number(); // object is accessed using the pointer

    return 0;
}
```

In C++ programming, **this** is a keyword that refers to the current instance of the class. There can be 3 main usages of this keyword in C++.

- It can be used to pass current object as a parameter to another method.
- It can be used to refer current class instance variable.
- It can be used to declare indexers.

VII Resources Required

Sr. No	Name Resource	Specification	Quantity	Remarks
1	Computer System	Any desktop or laptop computer with basic configuration	One computer system for each student	
2	Operating system	Windows /LINUX	One for each computer system	
3	Software	Turbo C++ Version 3.0 or any other	One for each computer system	

VIII Precautions to be followed

1. Handle computer system and peripherals with care.
2. Follow safety practices.

IX Exercise:

1. State output of the following code:

```
#include <iostream.h>
#include <conio.h>
class myclass
{
    int i;
public:
    void read(int j)
    {
        i = j;
    }
    int getint()
    {
        return i;
    }
};
void main()
{
    clrscr();
    myclass ob, *objectPointer;
```



```
objectPointer = &ob;
objectPointer->read(10);
cout<<objectPointer->getint();
getch();
```

2. Which is the pointer which denotes the object calling the member function?
 - a. Variable pointer
 - b) This pointer
 - c) Null pointer
 - d) Zero pointer
3. A pointer can be initialized with _____
 - a) Null
 - b) zero
 - c) Address of an object of same type
 - d) All of them
4. Write a program which show the use of this pointer.

C++ code:

Write "C++" Code for above exercise on the blank pages attached at the end of practical.

II Resources Used

Name of Resource	Suggested Broad Specification	Quantity
Computer system	desktop	1

III Result(s)

We have executed this program pointer to object this pointer.

III Conclusion

We have successfully completed pointer to object this pointer.

IV Practical Related Questions

Note: Below given are few sample questions for reference. Teacher must design more such questions so as to ensure the achievement of identified CO.

(Note: for all relevant programming exercise use blank pages provided or attach more pages if needed.)

- 1 Write a C++ program to declare a class "Book" containing data members

Object Oriented Programming Using C++ (313304)

- book_name, auther_name, and price . Accept this information for one object of the class using pointer to that object.
- 2 Write a C++ program to declare a class "Box" having data members height, width and breadth. Accept this information for one object using pointer to that object . Display the area and volume of that object.
3. Write a C++ program to declare a class birthday having data members day, month, year. Accept this information for five objects using pointer to the array of objects
4. Complete the following table:

Program Code	Write & Justify Output
<pre> a) #include <iostream> #include <iostream> using namespace std; class Employee { public: int id; //data member string name; //data member float salary; Employee(int id, string name, float salary) { this->id = id; this->name = name; this->salary = salary; } void display() { cout<<id<<" "<<name<<" "<<salary<<endl; } }; int main(void) { Employee e1 =Employee(101, "Sonoo", 890000); //creating an object of Employee Employee e2=Employee(102, "Nakul", 59000); //creating an object of Employee e1.display(); e2.display(); return 0; } </pre>	Enter rollno 11 Enter name "Promod" rollno is 11 name is Promod Justify:- The program prompts the user to enter a roll number.
<pre> b) #include <iostream> #include <string> using namespace std; class student { private: int rollno; string name; public: student():rollno(0),name(" ") { } student(int r, string n): rollno(r),name (n) { } void get() { cout<<"enter roll no"; cin>>rollno; cout<<"enter name"; cin>>name; } void print() { cout<<"roll no is </pre>	Output:- 101 Sonoo 890000 102 Nakul 59000


```

11 << rollno;
cout << "name is " << name;
};
void main ()
student *ps = new student; (*ps).get();
(*ps).print(); delete ps;

```

Justify :-
Print the details
of the first
employee.

Exercise :-

(Space for Answers)

- 1] 10% - Output of this program.
- 2] b] This pointer
- 3] d] All of the above

```

4] #include <iostream.h>
#include <conio.h>
class Rectangle
{
    int width, height;
public:
    Rectangle (int w, int h) {
        this->width = w;
        this->height = h;
    }
    int area() {
        return this->width * this->height;
    }
    void display() {
        cout << "width " << this->width << "Height " << this->height;
    }
};

void main()
{
    Rectangle rect (10, 5);
    rect.display();
    cout << "Area" << rect.area() << endl;
    getch();
}

```

Practical Related Questions

17

```
#include <iostream.h>
#include <string.h>
class Book
{
public:
    string book-name;
    string author-name;
    double price;
};

int main()
{
    Book book;    Book* bookptr = &book;
    cout << "enter book name";
    cout << "enter author name";
    cout << "enter price";
    cin >> bookptr -> price;
    cout << "Book Details";
    cout << "Book Name" << bookptr -> book-name << endl;
    cout << "Author Name" << bookptr -> author-name;
    return 0;
}
```

27

```
#include <iostream.h>
#include <conio.h>
class Box
{
    float height;
    float width;
    float breadth;
public:
    void acceptDimensions();
}
```



```
Cout << "enter height";  
Cin >> height;  
Cout << "enter width";  
Cin >> width;  
Cout << "enter breadth";  
Cin >> breadth;  
?
```

```
void displayArea () const {  
float area = 2 * (height * width + width * breadth +  
breadth * height);  
Cout << "Surface Area" << area << endl;  
?
```

```
void displayVolume () const {  
float volume = height * width * breadth;  
Cout << "volume" << volume << endl;  
?  
?}
```

```
int main() {  
Box* box pointer = new Box;  
box pointer -> acceptDimension();  
box pointer -> displayArea();  
box pointer -> displayVolume();  
delete box pointer;  
return 0;  
?
```

```
3] #include <iostream.h>
```

```
#include <conio.h>
```

```
class Birthday {
```

```
public:
```

```
int day;
```

```
int month;
```

```
int year;
```

```
};
```

```
int main() {
```

```
Birthday b[5];
```

```
Birthday *ptr = b;
```

```
for (int i=0; i<5; i++) {
```

```
cout << "enter day, month, and year for birthday" << i+1 << endl;
```

```
cin >> ptr->day >> ptr->month >> ptr->year;
```

```
ptr++;
```

```
}
```

```
cout << "Birthday" << endl;
```

```
ptr = b;
```

```
for (int i=0; i<5; i++) {
```

```
cout << "Birthday" << i+1 << "ptr->day" << i << ptr->month << "/" << ptr->year << endl;
```

```
ptr++;
```

```
}
```

```
return 0;
```

```
}
```


Practical No.17: Write programs for-

- Pointer to derived class in single inheritance
- Pointer to derived class in multilevel inheritance

I Practical Significance:

The key feature of class inheritance is that a pointer to a derived class is type-compatible with a pointer to its base class.

II Industry/Employer Expected Outcome(s)

This practical is expected to develop the following skills as:

1. Define pointer to derived class in single inheritance.
2. Define pointer to derived class in multilevel inheritance

III Course Level Learning Outcome(s)

Implement Polymorphism in C++.

IV Laboratory Learning Outcome(s)

Implement program to use pointer to derived class.

V Relevant Affective Domain related outcome(s)

1. Select proper programming environment in C++.
2. Follow ethical practices.

VI Relevant Theoretical Background

C++ allows base class pointers to point to derived class objects.

Let we have-

```
class base { ... };
```

class derived : public base { ... }; Then we can write -

```
base *p1;
```

```
derived d_obj;
```

```
p1=&d_obj;
```

```
base *p2 = new derived;
```

II

Object Oriented Programming Using C++ (313304)

Using a base class pointer (pointing to a derived class object) we can access only those members of the derived object that were inherited from the base.

- It is different from the behaviour that Java shows.
- We can get Java-like behaviour using virtual functions.
- This is because the base pointer has knowledge only of the base class.
- It knows nothing about the members added by the derived class.

VII Resources Required

Sr. No	Name Resource	Specification	Quantity	Remarks
1	Computer System	Any desktop or laptop computer with basic configuration	One computer system for each student	
2	Operating system	Windows / LINUX	One for each computer system	
3	Software	Turbo C++ Version 3.0 or any other	One for each computer system	

VIII Precautions to be followed

1. Handle computer system and peripherals with care.
2. Follow safety practices.

IX Exercise:

1. State output of the following code:

```
#include<iostream.h>
class base {
public:
void show()
cout << "base\n";
};
class derived: public base
public:
void show() {
cout << "derived\n";
};
void main()
{
base bl;
bl.show();
}
```


Object Oriented Programming Using C++ (313304)

```
derived dl;
dl.show();
base *pb = &bl;
pb->show();
pb = &dl;
pb->show();
}
```

2. A pointer to the base class can hold address of
 - A) only base class object
 - B) only derived class object
 - C) base class object as well as derived class object
 - D) None of the above
3. A pointer can be initialized with _____

- a) Null
- b) zero
- c) Address of an object of same type
- d) All of them

3. Which variable stores the memory address of another variable?
 - A) Reference
 - B) Pointer
 - C) Array
 - D) None of the above

C++ code:

Write "C++" Code for above exercise on the blank pages attached at the end of practical.

XI Resources Used

Sr. No.	Name of Resource	Suggested Broad Specification	Quantity
1	Computer system	desktop	1

XII Result(s)

we have executed this program single inheritance / multilevel inheritance.

XIII Conclusion

we have successfully completed pointer to derived classes.

XIV Practical Related Questions

Note: Below given are few sample questions for reference. Teacher must design more such questions so as to ensure the achievement of identified CO. (Note: for all relevant programming exercise use blank pages provided or

attach more pages if needed.)

1. Write a C++ program declare a class "polygon" having data members width and height. Derive classes "rectangle" and "triangle" from "polygon" having area() as a member function. Calculate area of triangle and rectangle using pointer to derived class object.
2. Write a program which show the use of Pointer to derived class in multilevel inheritance.
3. Complete the following table:

Program Code	Write & Justify Output
<pre> a) #include<iostream.h> class base { public: int n1; void show() { cout<<"n1"<<n1; } }; class derive:public base { public: int n2; void show() { cout<<"n1 "<<n1; cout<<"n2 "<<n2; } }; int main() { base b; base *bptr; cout<<"Pointer of base class points to it"; bptr=&b; bptr->n1=44; bptr->show(); derived d; cout<<"n"; bptr=&d; bptr->n1=66; bptr->show(); return 0; } </pre>	Pointer of base class points to it n1 44 n2 66

Object Oriented Programming Using C++ (313304)

```

b) #include <iostream.h>
class BaseClass {
int x;
public:
void setx(int i)
X = i;
int getx()
return x;
};

class DerivedClass : public BaseClass
{
int y;
public:
void sety(int i)
{
y = i;
}

int gety()
{
return y;
}
};

int main()
{
BaseClass *p;
BaseClass baseObject;
DerivedClass derivedObject;

p = &baseObject;
p->setx(10);
cout << "Base object x: " << p->getx() << "\n" ;
p = &derivedObject;
p->setx(99);
derivedObject.sety(88);
cout << "Derived object x: " << p->getx() << "\n" ;
cout << "Derived object y: " << derivedObject.gety() << "\n";
return 0;
}
    
```

Base object x: 10;
Derived object x: 99;
Derived object y: 88

(Space for Answers)

Exercise
Q.1] Base
Derived
Base
base

Q.2] c] base class object as well as derived class object

Q.3] d] All of them

Q.4] B] pointer

XV References/Suggestions for further reading

1. <https://www.scaler.com/topics/pointer-to-object-in-cpp/>
2. <https://www.geeksforgeeks.org/base-class-pointer-pointing-to-derived-class-object-in-cpp/>

XVI Assessment Scheme

Performance Indicators		Weightage
Process Related: 30 Marks		60 %
1	Logic formation	10%
2	Appropriate use of arithmetic expressions, operators and basic Input/output function.	20%
3	Debugging ability	20%
4	Follow ethical practices.	10%
Product Related: 20 Marks		40%
5	Expected Output	20%
6	Submitting the Manual in time	10%
7	Answer to sample questions	10%
Total (50 Marks)		100%

Marks Obtained			Dated signature of Teacher
Process Related (30)	Product Related (20)	Total (50)	

Practical Related Question

13 #include <iostream.h>

#include <conio.h>

class polygon {

protected:

double width, height;

public:

void setDimensions (double w, double h) {

width = w;

height = h;

}

};

class Rectangle : public polygon {

public:

double area() {

return width * height;

}

};

class triangle : public polygon {

public:

double area() {

return 0.5 * width * height; }

int main () {

Polygon * polyPtr;

Rectangle rect;

Triangle tri;

polyPtr = ▭

polyPtr -> setDimensions (5.0, 3.0);

cout << "Area of rectangle" << rect.area() << endl;

polyPtr = &tri;

polyPtr -> setDimensions (4.0, 6.0);

cout << "Area of triangle" << tri.area() << endl;

return 0; }

Practical No.18: Write programs which show the use of function overloading

I Practical Significance:

The concept of function overloading helps to use more than one definition for a function name in the same scope and helps to extend the concept of polymorphism.

II Industry/Employer Expected Outcome(s)

This practical is expected to develop the following skills as:

1. Define more than one definition for a function name in the same scope to implement function overloading.

III Course Level Learning Outcome(s)

Implement Polymorphism in C++.

IV Laboratory Learning Outcome(s)

Implement function overloading in a program.

V Relevant Affective Domain related outcome(s)

1. Select proper programming environment in C++.
2. Follow ethical practices.

VI Relevant Theoretical Background
Function overloading.

- Function overloading is a feature in C++ where two or more functions can have the same name but different parameters.
- The C++ compiler selects the proper function by examining the number, types and order of the arguments in the call.
- Function overloading is commonly used to create several functions of the same name that perform similar tasks but on different data types.
- Function overloading can be considered as an example of polymorphism feature in C++.
- There are two ways to overload the method in C++
 - a. By changing number of arguments or parameters
 - b. By changing the data type

Syntax

```
class class_Name
{
    returntype method()
```



```
int X = 5, y = 2; float n = 5.0,
m = 2.0;
cout << operate(x, y) << "\t"; cout
<< operate (n, m);
return 0;
```

3. Which of the following in Object Oriented Programming is supported by Function overloading and default arguments features of C++?

- a) Inheritance b) Polymorphism
c) Encapsulation d) None of these

4. Overloaded functions are

- a) Very long functions that can hardly run
b) One function containing another one or more functions inside it.
c) Two or more functions with the same name but different number parameters or type.
d) None of these

X C++ code:

Write "C++" Code for above exercise on the blank pages attached at the end of practical.

XI Resources Used

Sr. No.	Name of Resource	Suggested Broad Specification	Quantity
1.	Computer System	desktop any	1

XII Result(s)

we have executed this program function.....
overloading.....

XIII Conclusion

we have successfully completed of function.....
overloading.....

XIV

Practical Related Questions

Note: Below given are few sample questions for reference. Teacher must design more such questions so as to ensure the achievement of identified CO.

(Note: for all relevant programming exercise use blank pages provided or attach more pages if needed.)

1. Write a C++ Program to interchange the values of two int , float and char using function overloading.
2. Write a C++ Program that find the sum of two int and double number using function overloading.
3. Write C++ program to find the area of various geometrical shapes by Function overloading. (eg.Area of circle, circumference of circle etc...)
4. Complete the following table:

Program Code	Write & justify Output
<pre> a) #include<iostream.h> int absolute(int); float absolute(float); { int main() int a= -5; float b = 5.5; cout << "Absolute value of"<< a<<" " << absolute(a) << endl; cout << "Absolute value of"<< b <<" " << absolute(b); return 0; } int absolute(int var) { if (var < 0) { var= -var; return var; } float absolute(float var) { if (var< 0.0) { var= -var; return var; } } </pre>	Absolute value of -5 is 5 Absolute value of 5.5 is 5.5
<pre> b) #include <iostream.h> class Test { public: int main(int s) { cout << s << "\n"; return 0; } } </pre>	3 19ke C++ 96


```

}
int main(char *s)
{
cout << s << endl;
return 0;
}
int main(int s, int m)
{
cout << s << " " << m;
return 0;
};
int main()
{
Test obj;
obj.main(3);
obj.main("! like C++");
obj.main(9, 6);
return 0;
}
    
```

(Space for Answers)

Exercise:-

- Q1] Yes, Constructor overloading is a specific implementation of function overloading.
- Q2] 10. 2.5
- Q3] b] Polymorphism:- Is supported by function overloading and default arguments in object oriented programming (OOP).
- Q4] ☐ Two or more functions with the same name but different number of parameters.

Practical Related question

```
Q.1] #include <iostream.h>
#include <conio.h>
void interchange (int &x, int &y) {
    int temp = x;
    x = y;
    y = temp;
}
```

```
void interchange (float &x, float &y) {
    float temp = x;
    x = y;
    y = temp;
}
```

```
int main() {
    int a = 10, b = 20;
    float c = 3.14, d = 2.718;
    char e = 'A', f = '\0';
    cout << "Before interchanging";
    cout << "a=" << a << "b=" << b << endl;
    cout << "c=" << c << "d=" << d << endl;
    cout << "e=" << e << "f=" << f << endl;
    return 0;
}
```


Q.2] #include <iostream.h>
#include <conio.h>
int sum (int a, int b) {
 return a+b;
}
double sum (double a, double b) {
 return a+b;
}
int main () {
 int x=5, y=10;
 double m=3.14, n=2.718;
 int intsum = sum(x, y);
 double doublesum = sum(m, n);
 cout << "Sum of integer" << intsum << endl;
 cout << "Sum of double" << doublesum << endl;
 return 0;
}

Q.2] #include <iostream.h>
#include <conio.h>
int sum (int a, int b) {
 return a+b;
}
double sum (double a, double b) {
 return a+b;
}
int main () {
 int x=5, y=10;
 double m=3.14, n=2.718;
 int intsum = sum(x, y);
 double doublesum = sum(m, n);
 cout << "Sum of integer" << intsum << endl;
 cout << "Sum of double" << doublesum << endl;
 return 0;
}

break;

Case 3:

cout << "enter the length and breadth of
the rectangle";

cin >> length >> breadth;

cout << "Area the rectangle " << area << endl;

break;

default:

cout << "Invalid choice";

?

getch();

?



Practical No.19: Write programs to overload unary operator using-

- Member function
- Friend function

I Practical Significance:

The concept of operator overloading helps to assign the new meaning to the existing operator and helps to extend the concept of polymorphism.

II Industry/Employer Expected Outcome(s)

This practical is expected to develop the following skills as:

1. provide additional meaning to the unary operator using member function and friend function.
2. Define and use the overloaded operator function in the class.

III Course Level Learning Outcome(s)

Implement Polymorphism in C++.

IV Laboratory Learning Outcome(s)

1. Implement unary operator overloading using member function.
2. Implement unary operator overloading using friend function.

V Relevant Affective Domain related outcome(s)

1. Select proper programming environment in C++.
2. Follow ethical practices.

VI Relevant Theoretical Background

Operator overloading

Operator overloading is a type of polymorphism in which an operator is overloaded to give user defined meaning to it. The main purpose of operator overloading is to perform operation on user defined data type. For eg. The '+' operator can be overloaded to perform addition on various data types. Operator overloading is used by the programmer to make a program clearer. It is an important concept in C++.

Syntax:

```
Return_type classname :: operator OperatorSymbol (Argument_List)
{
```

```
//Statements:
```


}
The operator keyword is used for overloading the operators.

There are a few operators which cannot be overloaded are follows.

1. Scope resolution operator (::)
2. sizeof
3. member selector (.)
4. member pointer selector (*)
5. ternary operator (?)

There are some restrictions considered while implementing the operator overloading.

1. The number of operands cannot be changed. Unary operator remains unary, binary remains binary etc.
2. Only existing operators can be overloaded.
3. The precedence and associativity of an operator cannot be changed.
4. Cannot redefine the meaning of a procedure.

The Unary Operators can be overloaded by using:

1. Member Function
2. Friend Function

- In the case of a non-static member function, the unary operator should not have an argument.
- In the case of a friend function, the unary operator should have only one argument.
- They perform various operations such as incrementing/decrementing a value by one, negating an expression, or inverting the value of a boolean

▪ **Unary Operator Overloading Algorithm/Steps:**

- Step 1: Start the program.
- Step 2: Declare the class.
- Step 3: Declare the variables and its member function.
- Step 4: Using the function getvalue() to get the two numbers.
- Step 5: Define the function operator ++ to increment the values
- Step 6: Define the function operator -- to decrement the values.
- Step 7: Define the display function.
- Step 8: Declare the class object.
- Step 9: Call the function getvalue()
- Step 10: Call the function operator++ () by incrementing the class object and call the function display.
- Step 11: Call the function operator -- () by decrementing the class object and call the function display.
- Step 12: Stop the program.

VII Resources Required

Sr. No	Name Resource	Specification	Quantity	Remarks
1	Computer System	Any desktop or laptop computer with basic configuration	One computer system for each student	
2	Operating system	Windows /LINUX	One for each computer system	
3	Software	Turbo C++ Version 3.0 or any other	One for each computer system	

VIII Precautions to be followed

1. Handle computer system and peripherals with care.
2. Follow safety practices.

IX Exercise:

1. What is function overloading and operator overloading?
2. State output of the following code:

```
#include <iostream.h> class
Distance {
    private:
        int feet; int
        inches;

    public:
        // required constructors Distance()
        {
            feet= 0;
            inches= 0;
        }

        Distance(int f, inti)
        {
            feet= f;
            inches= i;
        }

        void displayDistance() {
            cout << "F: " << feet << " I: " << inches << endl;
        }

        // overloaded minus (-) operator Distance
        operator - () {
            feet= -feet; inches=
            -inches;
            return Distance(feet, inches);
        }
};

int main()
{
    Distance D1(11, 10), D2(-5, 11);
    -D1;
    D1.displayDistance();
}
```


Object Oriented Programming Using C++ (313304)

```
-D2;
D2.displayDistance();
return 0;
}
```

2. Overload the unary operator -(minus) using member function and friend function

X C++ code:

Write "C++" Code for above exercise on the blank pages attached at the end of practical.

XI Resources Used

Sr. No.	Name of Resource	Suggested Broad Specification	Quantity
1.	Turbo C++	Version 3.0	1

XII Result(s)

In this practical we learnt about concept of operator overloading.

XIII Conclusion

We successfully implemented code of operator overloading using member function.

XIV Practical Related Questions

Note: Below given are few sample questions for reference. Teacher must design more such questions so as to ensure the achievement of identified CO.

(Note: for all relevant programming exercise use blank pages provided or attach more pages if needed.)

Write a C++ program to overload unary operators (++) increment and (--) decrement by using member function and friend function.

1. Write C++ program to find the area of various geometrical shapes by function overloading. (eg. Area of circle, circumference of circle etc...)

Program Code	Write & justify Output
<pre> a) #include <iostream.h> class 3D { int x, y, z; public: 3D (int a=0, int b=0, int c=0) { x=a; y=b; z=c; } 3D operator++() { X=X+1; y=y+1; z=z+1; return *this; } 3D operator++(int) { 3D t = *this; X=X+1; y=y+1; z=z+1; return t; } 3D show() { cout<<"The elements are:\n" cout<<"x:"<<this->x<<" y:<<this->y<<"z:"<<this->z; } int main() { 3D pt1(2,4,5), pt2(7,1,3); cout<<"Point one's dimensions before increment are:"<< pt1.show(); ++pt1; cout<<"Point one's dimensions after increment are:"<< pt1.show(); cout<<"Point two's dimensions before increment are:"<< pt2.show(); pt2++; } </pre>	Output:- Point one dimensions before increments are: the elements are: x:2, y:4, z:5 Point one dimension after increment are: The elements are: x:3, y:5, z:6 Point two dimension before increments are: x:7, y:1, z:3 Point two dimension after increment are: The elements are: x:3, y:2, z:4

Exercise 2

Q.1] Ans:-

- Function Overloading:- It is to define multiple functions with the same name but different parameters.

Syntax:- return-type function-name (parameter);

- Operator Overloading:- It lets you redefine how standard operators (+, -, *, / etc.) work with user-defined types (like classes).

Syntax:-

class classname {

public:

return type operator op (argument list)

// implementation of the operator.

}

Q.2] Ans:-

Output:-

F: -11 I: -10

F: 5 I: -11

- After applying the unary minus operator, the signs of the feet and inches from invested both D1 & D2.
- Therefore, D1 changes from (11, 10) to (-11, -10) and D2 changes from (-5, 11) to (5, -11).

Q.3] #include <iostream.h>

#include <conio.h>

class Distance {

private:

int feet;

int inches;

public:

Distance () : feet (0), inches (0) {}

Distance (int f, int i) : feet (f), inches (i) {}

Distance operator - () const {}

return Distance (-feet, -inches);

}

Distance D1 (11,10) D2 (-5,11);

Distance D3 = D1;

D3->displayDistance();

Distance D4 = D2;

D4->displayDistance();

return 0;

}

Practical Related Questions

Q.11 #include <iostream.h>

class Counter {

private:

int Count;

Public:

Counter () { Count (0); }

Counter (int c) { Count (c); }

Counter Operator ++ () {

++ Count;

return * this;

}

void display () const {

Cout << "Count" << Count << endl;

}

};

int main () {

Counter c1 (5);

++ c1;

c1.display ();

-- c1;

c1.display ();

return 0;

}

Q.2] #include <iostream.h>

#include <conio.h>

#include <math.h>

class Geometry {

public:

double area (double radius) {

return $M - \pi * radius * radius$;

}

double circumference (double radius) {

return $2 * M - \pi * radius$;

}

double area (double length, double height) {

}

return length * breadth;

}

double area (double base, double height, char triangle) {

return $0.5 * base * height$;

};

int main() {

Geometry g;

double radius = 5.0;

double length = 10.0, breadth = 6.0;

double base = 8.0, height = 4.0;

cout << "Area of Circle " << g.area << endl;

cout << "Circumference " << g.circumference << endl;

cout << "Area of rectangle " << g.(length, breadth);

cout << "Area triangle " << g.area (base, height);

return 0;

}

Practical No.20: Write programs to overload binary operator using-

- Member function
- Friend function

I Practical Significance:

The concept of operator overloading helps to assign the new meaning to the existing operator and helps to extend the concept of polymorphism.

II Industry/Employer Expected Outcome(s)

This practical is expected to develop the following skills as:

1. provide additional meaning to the unary operator using member function and friend function.
2. Define and use the overloaded operator function in the class.

III Course Level Learning Outcome(s)

Implement Polymorphism in C++.

IV Laboratory Learning Outcome(s)

1. Implement binary operator overloading using member function.
2. Implement binary operator overloading using friend function.

V Relevant Affective Domain related outcome(s)

1. Select proper programming environment in C++.
2. Follow ethical practices.

VI Relevant Theoretical Background

Binary Operator Overloading:

An operator which contains two operands to perform a mathematical operation is called the Binary Operator Overloading. It is a polymorphic compile technique where a single operator can perform various functionalities by taking two operands from the programmer or user. There are multiple binary operators like +, -, *, /, etc., that can directly manipulate or overload the object of a class.

For example, suppose we have two numbers, 5 and 6; and overload the binary (+) operator. So, the binary (+) operator adds the numbers 5 and 6 and returns 11. Furthermore, we can also perform subtraction, multiplication, and division operation to use the binary operator for various calculations.

Syntax of the Binary Operator Overloading using member function

Following is the Binary Operator Overloading syntax in the C++ Programming language.

```
return_type :: operator binary_operator_symbol (arg)
{
    // function definition
}
```

Here,

return_type: It defines the return type of the function.

operator: It is a keyword of the function overloading.

binary_operator_symbol: It represents the binary operator symbol that overloads a function to perform the calculation.

arg: It defines the argument passed to the function.

Operator Overloading with Friend Functions:

Another method for overloading operators with non-member functions designated as friends inside the class is friend functions. Although friend functions are not class members, they can access the class's private members.

The general syntax for utilizing a friend function to overload an operator is as follows:

```
friend return_type operator op(parameters);
```

The function is declared a 'friend' of the class with this syntax, enabling it to access its private members. The remaining portions of the grammar resemble member function overloading.

Binary Operator Overloading Algorithm/Steps:

- Step 1: Start the program.
- Step 2: Declare the class.
- Step 3: Declare the variables and its member function.
- Step 4: Using the function get value() to get the two numbers.
- Step 5: Define the function operator +() to add two complex numbers.
- Step 6: Define the function operator -() to subtract two complex numbers.
- Step 7: Define the display function.
- Step 8: Declare the class objects obj1, obj2 and result.
- Step 9: Call the function get value using obj1 and obj2
- Step 10: Calculate the value for the object result by calling the function operator+ and operator-.
- Step 11: Call the display function using obj1 and obj2 and result.
- Step 12: Return the values.
- Step 13: Stop the program

VII Resources Required

Sr. No	Name Resource	Specification	Quantity	Remarks
1	Computer System	Any desktop or laptop computer with basic configuration	One computer system for each student	
2	Operating system	Windows /LINUX	One for each computer system	
3	Software	Turbo C++ Version 3.0 or any other	One for each computer system	

VIII Precautions to be followed

1. Handle computer system and peripherals with care.
2. Follow safety practices.

IX Exercise:

1. What is the difference between binary operator overloading with member function and binary operator overloading with friend function in C++? State output of the following code:

```
#include <iostream.h>
class Distance {
private:
int feet;
inches;
public:
// required constructors Distance() {
feet= 0;
inches= 0;
Distance(int f, inti)
{
feet= f;
inches= i;
}

void displayDistance() {
cout << "F: " << feet << " I: " << inches << endl;

// overloaded minus (-) operator Distance operator- ()
{
feet= -feet;
inches= -inches;
return Distance(feet, inches);
};
int main()
Distance D1(11, 10), D2(-5, 11);
-D1;
D1.displayDistance();
```



```

-D2;
D2.displayDistance();
return 0;
}
class overloading {
int value;
public:

void setValue(int temp) {
value= temp;
overloading operator+(overloading ob)
{
overloading t;
t.value =value+ ob.value;
return (t);
}

void display()
{
cout <<value<< endl;
}
};

void main() {
overloading obj1, obj2, result;
int a, b;
cout << "Enter the value of Complex Numbers a,b:";
cin >> a>>b;
obj1.setValue(a);
obj2.setValue(b);
result= obj1 + obj2;
cout << "Input Values:\n";
obj1.display();
obj2.display();
cout << "Result:";
result.display();
getch();
}

```

X C++ code:

Write "C++" Code for above exercise on the blank pages attached at the end of practical.

XI Resources Used

Sr. No.	Name of Resource	Suggested Broad Specification	Quantity
1.	Computer System	any desk-top	1

XII Result(s)

.....we have executed this program overload
binary operator.....

XIII Conclusion

.....we have successfully completed overload
binary operator.....

XIV Practical Related Questions

Note: Below given are few sample questions for reference. Teacher must design more such questions so as to ensure the achievement of identified CO.

(Note: for all relevant programming exercise use blank pages provided or attach more pages if needed.)

1. Write a C++ program to add two complex numbers using operator overloading by a friend function.
2. Write a C++ program to subtract two complex numbers using operator overloading by using member function.
3. Write a C++ program to compare two strings using '=' operator overloading.
4. Complete the following table:

Program Code	Write & justify Output
a) // Write a program to demonstrate binary operator overloading <pre>#include <iostream> using namespace std; class complex { int a, b; public: void get_data(){ cout << "Enter the value of Complex Numbers a, b: "; cin >> a >> b; } complex operator+(complex ob) // overaloded operator</pre>	output:- enter the value of complex number a,b:51 enter the value of complex number a,b:32 input values:


```

function +
{
    complex t;
    t.a = a + ob.a;
    t.b = b + ob.b;
    return (t);
}

complex operator-(complex ob)// overaloded operator
function -
{
    complex t;
    t.a = a - ob.a;
    t.b = b - ob.b;
    return (t);
}

void display() {
    cout << a << "+" << b << "i" << "\n";
}

int main()
{
    complex obj1, obj2, result, result1;
    obj1.get_data();
    obj2.get_data();

    result = obj1 + obj2;
    result1 = obj1 - obj2;

    cout << "\n\nInput Values:\n";

    obj1.display();
    obj2.display();

    cout << "\nResult:";

    result.display();
    result1.display();

    return 0;
}

```

$5+7i$, $3+2i$
 $\text{result} = 8+9i$
 $2+5i$

(Space for Answers)

Practical Related Question

```
Q.17 #include <iostream.h>
#include <conio.h>
class complex{
public:
    double real, imag;
    complex (double r=0.0, double i=0.0): real(r),
    imag(i){}
    friend complex operator + (const & c1, const & c2){
        void print() {
            cout << "real << " << "t" << imag << "i" << endl;
        }
        complex operator (const complex & c1, const & c2){
            complex result;
            result.real = c1.real + c2.real;
            result.imag = c1.imag + c2.imag;
            result.print();
        }
    }
    int main() {
        complex c1(2,3), c2(4,5);
        complex c3 = c1 + c2;
        c3.print();
        return 0;
    }
}
```



```

Q.2] #include <iostream.h>
#include <conio.h>
class Complex {
public:
    double real, imag;
    Complex (double r=0.0, double i=0.0) : real(r), imag(i) {}
    Complex operator - (const Complex & other) {
        Complex result;
        result.real = this->real - other.real;
        result.imag = this->imag - other.imag;
        return result;
    }
    void print () {
        cout << "real << " << real << "imag << " << imag << endl;
    }
};

int main () {
    Complex c1 (5, 3), c2 (2, 1);
    Complex c3 = c1 - c2;
    c3.print ();
    getch ();
}

```

Practical No.21: Write programs to implement virtual function

I Practical Significance:

The concept of virtual function helps to learn the concept of run time polymorphism in C++.

II Industry/Employer Expected Outcome(s)

This practical is expected to develop the following skills as:

1. Implementation of function overriding
2. Same function can be used for different purpose at run time.

III Course Level Learning Outcome(s)

Implement Polymorphism in C++.

IV Laboratory Learning Outcome(s)

Develop program using virtual function.

V Relevant Affective Domain related outcome(s)

1. Select proper programming environment in C++.
2. Follow ethical practices.

VI Relevant Theoretical Background:

Virtual Function:

A virtual function (also known as virtual methods) is a member function that is declared within a base class and is re-defined (overridden) by a derived class. When you refer to a derived class object using a pointer or a reference to the base class, you can call a virtual function for that object and execute the derived class's version of the method.

Virtual functions ensure that the correct function is called for an object, regardless of the type of reference (or pointer) used for the function call. They are mainly used to achieve Runtime polymorphism.

Functions are declared with a virtual keyword in a base class. The resolving of a function call is done at runtime.

Rules for Virtual Functions:

The rules for the virtual functions in C++ are as follows:

- Virtual functions cannot be static.
- A virtual function can be a friend function of another class.
- Virtual functions should be accessed using a pointer or reference of base class type to achieve runtime polymorphism.
- The prototype of virtual functions should be the same in the base as well as the derived class.

- They are always defined in the base class and overridden in a derived class. It is not mandatory for the derived class to override (or re-define the virtual function), in that case, the base class version of the function is used.
- A class may have a virtual destructor but it cannot have a virtual constructor.

Compile time (early binding) VS runtime (late binding) behaviour of Virtual Functions

Consider the following simple program showing the runtime behaviour of virtual functions.

```
// C++ program to illustrate
// concept of Virtual Functions
#include <iostream>
using namespace std;
class base
{
public:
    virtual void print()
    {
        cout << "print base class\n";
    }
    void show()
    {
        cout << "show base class\n";
    }
};

class derived : public base
{
public:
    void print()
    {
        cout << "print derived class\n";
    }
    void show()
    {
        cout << "show derived class\n";
    }
};

int main()
{
    base* bptr;
    derived d;
    bptr = &d;
    // Virtual function, binded at runtime
    bptr->print();
    // Non-virtual function, binded at compile time
    bptr->show();
}
```

```

        return 0;
    }

```

VII Resources Required

Sr. No	Name Resource	Specification	Quantity	Remarks
1	Computer System	Any desktop or laptop computer with basic configuration	One computer system for each student	
2	Operating system	Windows / LINUX	One for each computer system	
3	Software	Turbo C++ Version 3.0 or any other	One for each computer system	

VIII Precautions to be followed

1. Handle computer system and peripherals with care.
2. Follow safety practices.

IX Exercise:

1. What is function overriding?
2. Write output for following code:

```

#include<iostream.h>
class A
{
public:
virtual void fun()
{
cout << "\n A::fun() called ";
}
};
class B : public A
{
public:
void fun()
{
cout << "\n B::fun() called ";
}
};
class C : public B
{
public:
void fun()
{
cout << "\n C::fun() called ";
}
};

```



```
int main()
{
    // An object of class C
    C c;
    // A pointer of class B pointing
    // to memory location of c
    B* b = &c;

    // this line prints "C::fun() called"
    b->fun();

    getch(); // to get the next character
    return 0;
}
```

3. There are the following statements that are given below, which of them are correct about virtual function in C++?

- A. The virtual function is the type of runtime polymorphism in C++.
- B. The virtual function is used in inheritance where a function is declared in the base and it is overridden in child class.
- C. For virtual function implementation we need did not require any special keyword.
- D. All of the above

Options:

- 1. A
- 2. A and B
- 3. A, B, and C
- 4. D

X C++ code:

Write "C++" Code for above exercise on the blank pages attached at the end of practical.

XI Resources Used

Sr. No.	Name of Resource	Suggested Broad Specification	Quantity
1.	Turbo c++	Version 3.0	1

XII Result(s)

In this practical we learn about how to virtual function works.

XIII Conclusion

we successfully executed program to implemented virtual functions.

XIV Practical Related Questions

Note: Below given are few sample questions for reference. Teacher must design more such questions so as to ensure the achievement of identified CO.

(Note: for all relevant programming exercise use blank pages provided or attach more pages if needed.)

1. Define a class parent in which use read function, define another class child use same read function. Display the data of both the read function on output screen using virtual function.
2. Write a program which shows the use of function overriding.
3. Complete the following table:

Program Code	Write & justify Output
<pre> a) #include <iostream> using namespace std; class Base { public: virtual void print() { cout << "Base Function" << endl; } }; class Derived : public Base { public: void print() override { cout << "Derived Function" << endl; } }; int main() { Derived derived1; // pointer of Base type that points to derived1 Base* base1 = &derived1; // calls member function of Derived class base1->print(); return 0; } </pre>	Output:- Derived function
<pre> b. #include <iostream> using namespace std; class Base { </pre>	


```
public:
    virtual void display()
    {
        cout << "Base class function";
    }
};
```

```
class Child : public Base {
public:
    virtual int display()
    {
        cout << "Child class function";
    }
};
```

```
int main()
{
    Child C;

    C.display();

    return 0;
}
```

output :-
child class
function

(Space for Answers)

IX. Exercise

Q.1] Ans:-

Function Overloading :- Function overloading is a type of polymorphism in which we predefine the member function of a class that is inherited from its base class.

Q.2] Ans:-

C::func() called.

Q.3] Ans:-

2. A and B

Q.2] Ans 8-

```
#include <iostream.h>
#include <conio.h>
class Base {
public:
    virtual void show() {
        cout << "Base class show function." << endl;
    }
};

class Derived : public Base {
public:
    void show() override {
        cout << "Derived class show function" << endl;
    }
};

void main() {
    Base base;
    Derived derived;
    Base * base_ptr;
    base_ptr = &base;
    base_ptr -> show();
    base_ptr = &derived;
    base_ptr -> show();
    getch();
}
```


XIV. Practical Related Questions.

Q.17 Ans 8-

```
#include <iostream.h>
```

```
#include <conio.h>
```

```
class parent {
```

```
public:
```

```
virtual void read () {
```

```
cout << "Reading data in Parent class" << endl;
```

```
}
```

```
};
```

```
class child : public parent {
```

```
public:
```

```
void read () override {
```

```
cout << "Reading data in child class" << endl;
```

```
}
```

```
};
```

```
void main () {
```

```
parent * ptr;
```

```
parent p;
```

```
child c;
```

```
ptr = &p;
```

```
ptr -> read ();
```

```
ptr = &c;
```

```
ptr -> read ();
```

```
getch ();
```

```
}
```

Practical No.22: Write programs to implement pure virtual function

I Practical Significance:

The concept of pure virtual function helps to learn the concept of run time polymorphism in C++.

II Industry/Employer Expected Outcome(s)

This practical is expected to develop the following skills as:
A pure virtual function is implemented by classes that are derived from an Abstract class.

III Course Level Learning Outcome(s)

Implement Polymorphism in C++.

IV Laboratory Learning Outcome(s)

Develop program using pure virtual function

V Relevant Affective Domain related outcome(s)

1. Select proper programming environment in C++.
2. Follow ethical practices.

VI Relevant Theoretical Background:

Pure Virtual Function:

Sometimes implementation of all functions cannot be provided in a base class because we don't know the implementation. Such a class is called an abstract class. For example, let Shape be a base class. We cannot provide the implementation of function draw() in Shape, but we know every derived class must have an implementation of draw(). Similarly, an Animal class doesn't have the implementation of move() (assuming that all animals move), but all animals must know how to move. We cannot create objects of abstract classes.

A pure virtual function (or abstract function) in C++ is a virtual function for which we can have an implementation. But we must override that function in the derived class, otherwise, the derived class will also become an abstract class. A pure virtual function is declared by assigning 0 in the declaration.

**Example of Pure Virtual Functions
C++**

```
// An abstract class
class Test {
    // Data members of class
public:
```



```
int main(void)
{
    // creating an object of Derived class
    Derived d;

    // calling the fun() function of Derived class
    d.fun();

    return 0;
}
```

```
3. #include <iostream>
using namespace std;
```

```
class Base {
public:
    virtual void show() = 0;
};
```

```
class Derived : public Base {
public:
    void show() { cout << "In Derived \n"; }
};
```

```
int main(void)
{
    Base* bp = new Derived();
    bp->show();
    return 0;
}
```

X C++ code:

Write "C++" Code for above exercise on the blank pages attached at the end of practical.

XI Resources Used

Sr. No.	Name of Resource	Suggested Broad Specification	Quantity
1	Turbo c++	Version 3.0	1

XII Result(s)

In this practical we learn about the concept of pure virtual function and also abstract class.

XIII Conclusion

We completed practical and implement the code pure virtual function.

XIV Practical Related Questions

Note: Below given are few sample questions for reference. Teacher must design more such questions so as to ensure the achievement of identified CO.

(Note: for all relevant programming exercise use blank pages provided or attach more pages if needed.)

1. Write a program which show the use of Abstract class.
2. Complete the following table:

Program Code	Write & justify Output
<pre> a) #include <iostream> using namespace std; class B { public: virtual void s() = 0; // Pure Virtual Function }; class D: public B { public: void s() { cout << "Virtual Function in Derived class\n"; } }; int main() { B *b; D dobj; b = &dobj; b->s(); } </pre>	Output: Virtual function in derived class.
<pre> b) #include <iostream.h> using namespace std; class Test { </pre>	


```

int x;

public:
    virtual void show() = 0;
    int getX() { return x; }
};

int main(void)
{
    Test t;
    return 0;
}

```

error: cannot
declare
variable 't'
of abstract
type 'Test'.

(Space for Answers)

IX Practical Related questions

Q.1] Ans:-

```

#include <iostream.h>
#include <conio.h>
class shape {
public:
    virtual void draw() = 0;
};
class circle : public shape {
public:
    void draw() override {
        cout << "Drawing circle" << endl;
    }
};
void main() {
    circle c;
    c.draw();
    getch();
}

```

IX. Exercises:-

Q.1] Ans:-

Pure virtual function:- Pure virtual function is a pure virtual function in the base class. The derived class overrides this function and provides its implementation.

Syntax:- class Base {

public:

virtual void pureVirtualFunction() = 0;

};

Q.2] Ans:- In Derived.

Practical No.23: Write programs to read and write from/to file using –

- Constructor
- Open()

I Practical Significance:

Files store the output of a program and help to perform various operations on it. Files help store these data permanently on a storage device.

II Industry/Employer Expected Outcome(s)

Virtual base classes used to prevent multiple instances of a given class from appearing in an inheritance hierarchy when using multiple inheritances.

III Course Level Learning Outcome(s)

Implement Read and Write operation From/to file in C++ program.

IV Laboratory Learning Outcome(s)

1. Write/ Compile/ debug/ Execute simple C++ program using read and write data from a file.

V Relevant Affective Domain related outcome(s)

1. Select proper programming environment in C++.
2. Follow ethical practices.

VI Relevant Theoretical Background

File Handling:

1. This concept in C++ language is used for store a data permanently in computer.
2. Using file handling we can store our data in Secondary memory (Hard disk).

Standard File handling Classes

1. Ofstream: This file handling class in C++ signifies the output file stream and is applied to create files for writing information to files.
2. Ifstream: This file handling class in C++ signifies the input file stream and is applied for reading information from files.
3. Fstream: This file handling class in C++ signifies the file stream generally, and has the capabilities for representing both ofstream and ifstream.

Opening a File:

1. The first operation generally performed on an object of one of these classes to use a file is the procedure known as to opening a file.
2. An open file is represented within a program by a stream and any input or output task performed on this stream will be applied to the physical file associated with it.
3. The syntax of opening a file in C++ is:

`open (filename, mode);`

4. There are some mode flags used for file opening.

`ios::app:` append mode

`ios::ate:` open a file in this mode for output and read/write controlling to the end of the file

`ios::m:` open file in this mode for reading

`ios::out:` open file in this mode for writing

`ios::trunc:` when any file already exists, its contents will be truncated before file opening

Closing a File:

1. When any C++ program terminates, it automatically flushes out all the streams releases all the allocated memory and closes all the opened files.
2. But it is good to use the `close()` function to close the file related streams and it is a member of `ifstream`, `ofstream` and `fstream` objects.
3. The structure of using this function is:

`void`

`close();` General functions used for

file handling:

1. `open();` To create a file
2. `close();` To close an existing file
3. `get();` to read a single character from the file
4. `put();` to write a single character in the file
5. `read();` to read data from a file
6. `write();` to write data into a file

VII Resources Required

Sr. No	Name Resource	Specification	Quantity	Remarks
1	Computer System	Any desktop or laptop computer with basic configuration	One computer system for each student	
2	Operating system	Windows / LINUX	One for each computer system	

3	Software	Turbo C++ Version 3.0 or any other	One for each computer system	
---	----------	------------------------------------	------------------------------	--

VIII Precautions to be followed

1. Handle computer system and peripherals with care.
2. Follow safety practices.

IX Exercise:

1. Write output for following code

```
#include <iostream>
#include <fstream>
using namespace std;
int main () {
    ofstream filestream("testout.txt");
    if (filestream.is_open())
    {
        filestream << "Welcome to javaTpoint.\n";
        filestream << "C++ Tutorial.\n";
        filestream.close();
    }
    else cout << "File opening is fail.";
    return 0;
}
```

2. Write a program in C++ in which open a file read and write mode and display the content on
3. output screen using function open()
4. Write a program in C++ in which open a file read and write mode and display the content on output screen using constructor.

X C++ code:

Write "C++" Code for above exercise on the blank pages attached at the end of practical.

XI Resources Used

Sr. No.	Name of Resource	Suggested Broad Specification	Quantity
1.	Computer system	Any desktop	1.

XII Result(s)

In this practical we learn the concept of file.

XIII Conclusion

We successfully completed program to read and write from the using constructor open().

XIV Practical Related Questions

Note: Below given are few sample questions for reference. Teacher must design more such questions so as to ensure the achievement of identified CO.

(Note: for all relevant programming exercise use blank pages provided or attach more pages if needed.)

- Which operator is used to insert the data into file?
a) >> b) <<
c) < d) none of the mentioned
- Which function is used to position back from the end of file object?
a) seekg() b) seekp()
c) both seekg() & seekp() d) none of the mentioned
- Write a C++ program to Read and Display File's Content.
- Write a C++ program to List Files in Current Directory.
- What is the return type open() method?
a) int
b) char
c) bool
d) float
- Complete the following table:

Program Code	Write & justify Output
<pre>a) #include <iostream.h> #include <fstream.h> #include <cctype.h> int main () { ifstream ifile; ifile.open ("text.txt"); char last; ifile.ignore (256, ' '); last= ifile.get(); cout << "Your initial is" << last << '\n'; ifile.close(); return 0 }</pre>	The first character after ignoring the first 256 characters.


```

b) #include <iostream>
#include <fstream>
using namespace std;
int main ()
{
    int length;
    char * buffer;
    ifstream is;
    is.open ("sample.txt", ios :: binary);
    is.seekg (0, ios :: end);
    length = is.tellg();
    is.seekg (0, ios :: beg);
    buffer = new char [length];
    is.read (buffer, length);
    is.close();
    cout.write (buffer, length);
    delete[] buffer;
    return 0;
}

```

This program reads the entire content of 'sample.txt' in binary mode and then writes it to the standard output.

(Space for Answers)

Q.1] Ans:- b) <<

Q.2] Ans:- C) both seekg () & seekp ()

Q.5] Ans:- C) bool

Q.3] Ans:-

```

#include <fstream.h>
#include <fstream.h>
#include <string.h>
int main () {
    if stream file ("example.txt");
    if (file.is-open()) {
        cout << "Error : could not open the file" << endl;
        return 1;
    }
    string line;
    cout << "file content : " << endl;
    while (getline(file, line)) {
        cout << line << endl;
    }
    file.close();
    return 0;
}

```


Q.4] Ans:-

```
#include <fstream.h>
#include <conio.h>
#include <filesystem.h>

int main() {
    string path = ".";
    cout << "Files in the current directory: " << endl;
    try {
        for (const auto & entry : fs::directory_iterator(path)) {
            if (fs::is_regular_file(entry.status())) {
                cout << entry.path().filename().string() << endl;
            }
        }
    } catch (const fs::filesystem_error & e) {
        cout << "Error: " << e.what() << endl;
    }
    return 0;
}
```

Exercise 8:-

1] Ans:-

The content of a text file testout.txt is set with the data: welcome to JavaTpoint.
C++ Tutorial.

Q.2] Ans:-

```
#include <fstream.h>
#include <fstream.h>
#include <conio.h>

int main() {
```

```

fstream file;
string line;
file.open("file.txt", ios::in | ios::out | ios::app);
if (!file) {
    cout << "Error opening file " << endl;
    return 1;
}
cout << "File content: " << endl;
while (getline(file, line)) {
    cout << line << endl;
}
file.clear();
file.seekp(0, ios::end);
file << "Adding a new line to the file";
file.close();
cout << "New content added successfully";
return 0;
}

```

Q.3] Ans:-

```

#include <iostream.h>
#include <fstream.h>
#include <conio.h>
void displayfile_content(const char* filename)
{
    fstream file;
    string line;
    file.open(filename, ios::in);
    if (!file) {
        cout << "Error opening file." << endl;
    }
    cout << "File content: " << endl;
}

```



```
while (getline (fFile, lPne)) {
```

```
    cout << lPne << endl;
```

```
}
```

```
fFile.close();
```

```
}
```

```
int main() {
```

```
    const char * file name = "file.txt";
```

```
    displayFileContent (filename);
```

```
    getch();
```

```
}
```

Q.47 Ans:-

```
#include <fstream.h>
```

```
#include <fstream.h>
```

```
#include <conio.h>
```

```
class FileHandler {
```

```
public:
```

```
    fstream file;
```

```
    FileHandler (const string & fName) {
```

```
        file.open (fName, ios::in | ios::out | ios::app);
```

```
        if (!file) {
```

```
            cout << "Error opening file " << endl;
```

```
            exit (1);
```

```
        }
```

```
    }
```

```
    void displayAndWrite() {
```

```
        string line;
```

```
        while (getline (file, line)) {
```

```
            cout << line << endl;
```

```
        }
```

```
file.clear();  
file.seekp(0, ios::end);  
file << "\n Adding a new line using a constructor";  
}
```

```
~fileHandler() {
```

```
file.close();
```

```
}
```

```
};
```

```
void main() {
```

```
fileHandler fh("file.txt");
```

```
fh.displayAndWrite();
```

```
cout << "New content added successfully"  
      << endl;
```

```
getch();
```

```
}
```



CAMPUSIFY

Practical No.24: Write programs to copy the contents of one file into another file using formatted input/output function.

I Practical Significance:

To copy the file using C++, we read the contents of the source file and write it into the destination file.

II Industry/Employer Expected Outcome(s)

Copying is a file operation that creates a new file which has the same content as an existing file

III Course Level Learning Outcome(s)

Implement copy the contents of one file into another file using formatted input/output Function in C++ program.

IV Laboratory Learning Outcome(s)

Write/ Compile/ debug/ Execute simple C++ program to copy the contents of one file into another file using formatted input/output function.

V Relevant Affective Domain related outcome(s)

1. Select proper programming environment in C++.
2. Follow ethical practices.

VI Relevant Theoretical Background

Formatted I/O in C++

C++ helps you to format the I/O operations like determining the number of digits to be displayed after the decimal point, specifying number base etc.

Example:

- If we want to add + sign as the prefix of our output, we can use the formatting to do so:
`stream.setf(ios::showpos)`
If input=100, output will be +100
- If we want to add trailing zeros in our output to be shown when needed using the formatting:
`stream.setf(ios::showpoint)`
If input=100.0, output will be 100.000

C++ Program to Copy One File into Another File

To copy the text/contents of one file to another file, we should know the basics of reading and writing a text file in C++. To copy the file using C++, we read the contents of the source file and write it into the destination file.

Steps to copy one file to another in C++:

1. Create objects of ifstream and ofstream classes.
2. Check if they are connected to their respective files. If so, go ahead otherwise check the filenames twice.
3. Read the contents of the source file using the getline() method and write the same to the destination using the << operator (i.e. copy each line from ifstream object to ofstream object).
4. Close files after the copy using the close() method.
5. End the program.

VII Resources Required

Sr. No	Name Resource	Specification	Quantity	Remarks
1	Computer System	Any desktop or laptop computer with basic configuration	One computer system for each student	
2	Operating system	Windows / LINUX	One for each computer system	
3	Software	Turbo C++ Version 3.0 or any other	One for each computer system	

VIII Precautions to be followed

2. Handle computer system and peripherals with care.
3. Follow safety practices.

IX Exercise:

1. Write programs to copy the contents of one file into another file using formatted input/output function

X C++ code:

Write "C++" Code for above exercise on the blank pages attached at the end of practical.

XI Resources Used

Sr. No.	Name of Resource	Suggested Broad Specification	Quantity
1.	Turbo C++	version 3.0	1

XII Result(s)

we efficiently copies content from one file to another using is stream of stream ensuring basic error handling.

XIII Conclusion

We successfully completed program to copy the file into another file.

XIV Practical Related Questions

Note: Below given are few sample questions for reference. Teacher must design more such questions so as to ensure the achievement of identified CO.

(Note: for all relevant programming exercise use blank pages provided or attach more pages if needed.)

- Write a C++ program to Read Content From One File and Write it Into Another File.
- Write a C++ Program to Copy the Contents of One File Into Another File.
- Complete the following table:

Program Code	Write & justify Output
a) What is the use of ios::trunc mode? a) To open a file in input mode b) To open a file in output mode c) To truncate an existing file to half d) To truncate an existing file to zero	d] To Truncate an existing file to zero.
b) Which of the following is not a file opening mode a. ios::ate b. ios::nocreate c. ios::noreplace d. ios::truncate	c] ios::noreplace

State output

Q.2] Ans:-

(Space for Answers)

```
#include <iostream.h>
#include <conio.h>
#include <fstream.h>
int main() {
    ifstream sourcefile("source.txt", ios::in);
    ofstream destfile("destination.txt", ios::out);
    if (!sourcefile || !destfile) {
        cout << "Error opening files." << endl;
        return 1;
    }
    char ch;
    while (sourcefile.get(ch)) {
        destfile.put(ch);
    }
    sourcefile.close();
    destfile.close();
}
```

```
cout << "File content copied
successfully." << endl;
return 0;
}
```

XV References/Suggestions for further reading

1. <https://www.geeksforgeeks.org/cpp-program-to-copy-one-file-into-another-file/>
2. <https://www.tutorialspoint.com/c-program-to-copy-the-contents-of-one-file-to-another-file>

XVI Assessment Scheme

Performance Indicators		Weightage
Process Related: 30 Marks		60%
1	Logic formation	10%
2	Appropriate use of File copy functions.	20%
3	Debugging ability	20%
4	Follow ethical practices.	10%
Product Related: 20 Marks		40%
5	Expected Output	20%
6	Submitting the Manual in time	10%
7	Answer to sample questions	10%
Total (50 Marks)		100 %

Marks Obtained			Dated signature of Teacher
Process Related (30)	Product Related (20)	Total (50)	

Exercise:-

```
#include <iostream.h>
#include <fstream.h>
#include <conio.h>
void copy_file_contents (const char * source_file,
const char * dest_file) {
    ifstream source (source_file, ios::in);
    ofstream destination (dest_file, ios::out);
    char ch;
    if (!source || !destination) {
        cout << "Error opening files." << endl;
        return;
    }
    while (source.get(ch)) {
        destination.put(ch);
    }
    source.close();
    destination.close();
    cout << "File copied successfully" << endl;
}

void main () {
    const char * source_file = "source.txt";
    const char * dest_file = "destination.txt";
    copy_file_contents (source_file, dest_file);
    getch();
}
```

Practical Related Questions.

Q.1] Ans:-

```
#include <iostream.h>
#include <fstream.h>
#include <conio.h>

int main () {
    ifstream sourcefile ("source.txt");
    ofstream destfile ("destination.txt");
    if (!sourcefile.is_open() || !destfile.is_open()) {
        cout << "Error opening files." << endl;
        return 1;
    }
    string line;
    while (getline (sourcefile, line)) {
        destfile << line << endl;
    }
    sourcefile.close();
    destfile.close();
    cout << "File Content copied successfully" << endl;
    return 0;
}
```